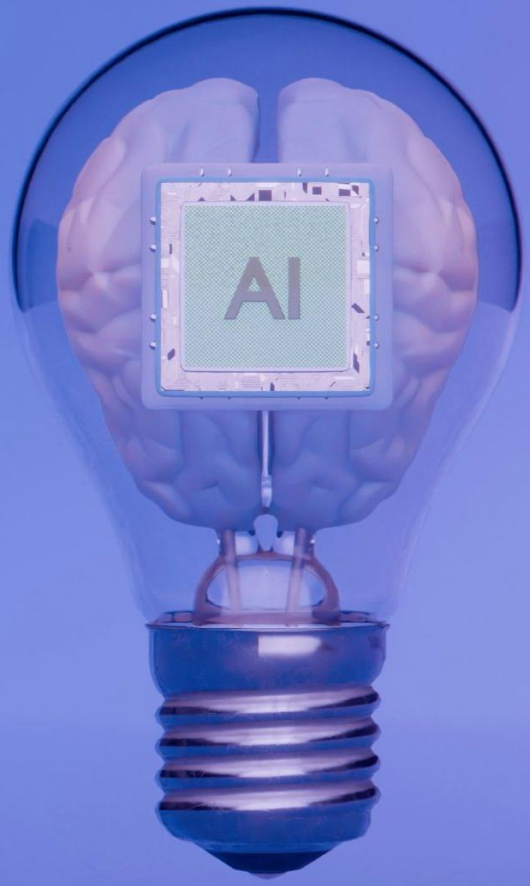
AI & IoT



Panwit Tuwanut


SYSTEM ONLINE
J.A.R.V.I.S.
VOICE RECOGNITION
OBJECT DETECTION
ENVIRONMENT ANALYSIS
SYSTEM DIAGNOSTICS
TARGET ANALYSIS
DISTANCE
ELEVATION
SPEED
THREAT LEVEL
LOCATION
GLOBAL SCAN
SATELLITE LINK
TERRAIN MAPPING
THREAT ANALYSIS
SYSTEM STA
REACTOR
SUIT INTEGR
POWER LEVE
AUXILIARY
COMMUNICAT
NAVIGATION

1 จุดเริ่มต้นของ AI : จากคำถามของ Alan Turing สู่การกำเนิดศาสตร์ AI

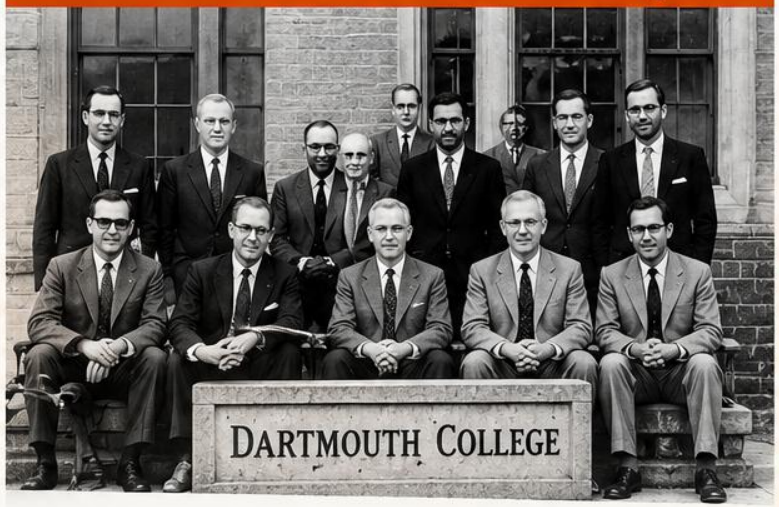
1950 Alan Turing
ตั้งคำถามที่เปลี่ยนโลก



“เครื่องจักรคิดได้ไหม”
(Can machines think?)

Alan Turing ตีพิมพ์บทความ
“**Computing Machinery and Intelligence**”
และตั้งคำถามที่เป็นรากฐานของทุกสิ่ง
ว่า**“เครื่องจักรคิดได้ไหม”**

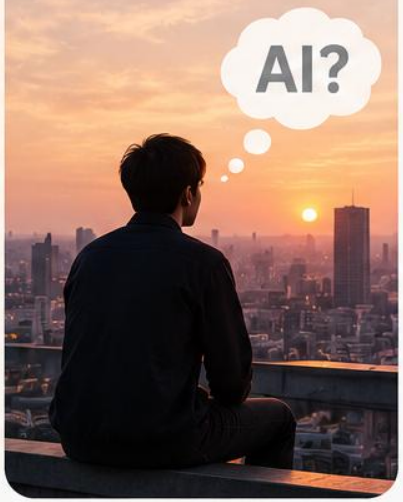
1956 Dartmouth Conference
กำเนิดศาสตร์ Artificial Intelligence



- กลุ่มนักวิชาการชั้นนำ นัดประชุมที่ Dartmouth College (ฤดูร้อนปี 1956)
- แสนอให้ใช้คำว่า **“Artificial Intelligence”**
- ถือเป็นจุดเริ่มต้นของศาสตร์ AI อย่างเป็นทางการ

ผ่านมาแล้วกว่า 70 ปี

AI อยู่กับเรามาตั้งแต่ปี 1956
แต่ในช่วงเวลานานหนั้น
แทบไม่มีใครสนใจมันเลย

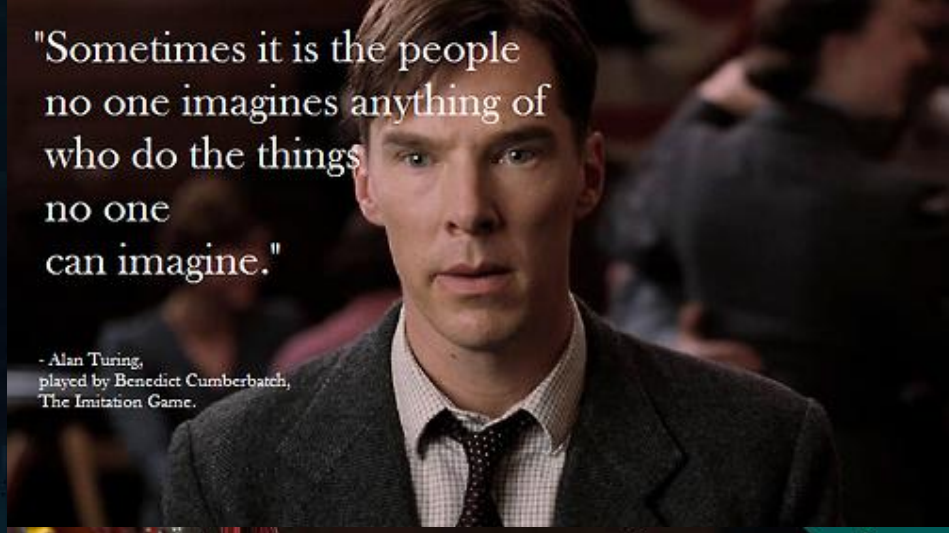


1950
Turing
ถามคำถามแรก

1956
Dartmouth Conference
กำเนิด AI อย่างเป็นทางการ

1956 – ปัจจุบัน
ผ่านมากว่า 70 ปี
แต่หลายคนยังไม่ให้ความสนใจ...







บางครั้งคนที่ไม่มีใครคาดหวัง
ก็อาจทำในสิ่งที่ไม่มีใคร “ คาดคิด ”

คำคมจากภาพยนตร์เรื่อง : The imitation game (2014)

สองแนวทางที่ฟาดฟันกันในยุคบุกเบิกของ AI

จากความหวังอันยิ่งใหญ่ สู่ฤดูหนาวของ AI และความพยายามของกลุ่มนักวิทยาศาสตร์ที่ไม่ยอมแพ้

สายตรรกะสัญลักษณ์ (Symbolic AI)

ฝ่ายเสียงข้างมาก กุมงบวิจัยเกือบทั้งหมด

- เชื่อว่าปัญญาของมนุษย์เกิดจากกฎเกณฑ์ทางตรรกะ
- ถ้าเขียนโค้ดตั้งเงื่อนไข If-Else ให้ละเอียดพอ คอมพิวเตอร์ก็จะฉลาดเหมือนคน
- พัฒนา Expert Systems และระบบที่อาศัยกฎเกณฑ์
- ได้รับความนิยมและงบประมาณวิจัยเป็นหลัก

```
IF องค์ประกอบ = "ขน"  AND
   หูแหลม   AND  หางยาว
   THEN  เป็น "แมว"
ELSE IF องค์ประกอบ = "หูตก"  AND
        ลิ้นยาว  AND  ตัวใหญ่
THEN   เป็น "หมา"
...
```


VS

แต่ความจริงกลับซับซ้อนเกินกว่ากฎเกณฑ์ ทฤษฎี Symbolic AI ล้มเหลวไม่เป็นท่า



เราแยกหมากับแมวได้ในเสี้ยววินาที โดยไม่ต้องนั่งวัดมุมกะโหลก



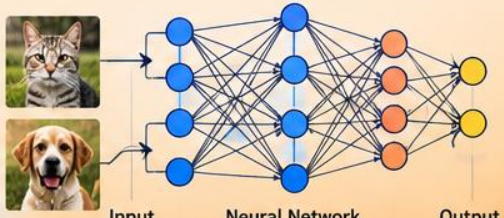


- เมื่อกฎเกณฑ์ไม่สามารถครอบคลุมความจริงอันยุ่งเหยิงของโลกได้
- วงการ AI จึงล่มสลายเข้าสู่ "ฤดูหนาวของ AI (AI Winter)"
- นักวิจัยโดนตัดทุน จนแทบไม่มีที่ยืน

โครงข่ายประสาทเทียม (Connectionism / Neural Networks)

กลุ่มนักวิทยาศาสตร์หัวรั้น ที่ไม่ยอมแพ้

- เสนอว่าไม่ควรสอนกฎเกณฑ์ แต่ควรลอกเลียนแบบโครงสร้างเซลล์ประสาทในสมองมนุษย์
- ให้เป็นโครงข่ายตัวเลขว่างเปล่า (Neural Network)
- เรียนรู้ความสัมพันธ์ของข้อมูลด้วยตัวมันเอง ผ่านการปรับค่าน้ำหนักทางคณิตศาสตร์ (Learning)
- ถูกวงการวิชาการหัวเราะเยาะ และตราหน้าว่า "ขายฝัน" มานานกว่า 30 ปี เพราะคอมพิวเตอร์ในเวลานั้นไม่มีแรงพอที่จะคำนวณโครงข่ายขนาดใหญ่ได้



นักวิทยาศาสตร์หัวรั้นที่ไม่ยอมแพ้







1950
Alan Turing
ตั้งคำถาม
"เครื่องจักรคิดได้ไหม"

1950s–1970s
Symbolic AI
ได้รับความนิยม
และงบประมาณวิจัย

1970s–1980s
AI Winter
วงการ AI ล่มสลาย
นักวิจัยถูกตัดทุน

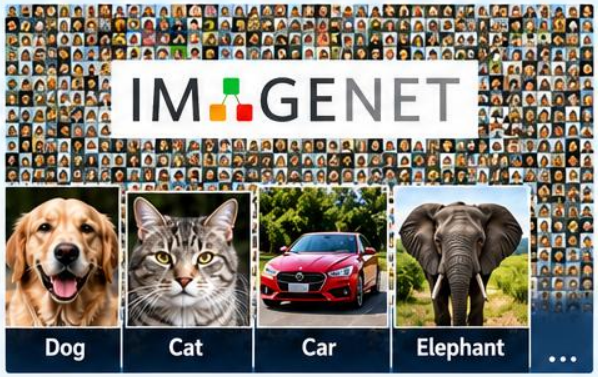
แม้จะถูกมองว่าเป็นพวกขายฝัน
แต่กลุ่มนักวิทยาศาสตร์ด้าน Neural Networks
ยังคงไม่ยอมแพ้ และเดินหน้าต่อไป

2012 ตุลาคม จุดเปลี่ยนครั้งสำคัญของประวัติศาสตร์ AI

AlexNet ชนะการแข่งขัน ImageNet ด้วยพลังของ GPU

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

การแข่งขันแยกประเภทภาพที่ใหญ่ที่สุดในโลก



ทีมของ Geoffrey Hinton นำ Deep Learning กลับมาลงสนาม โดยใช้ชิป GPU แทน CPU

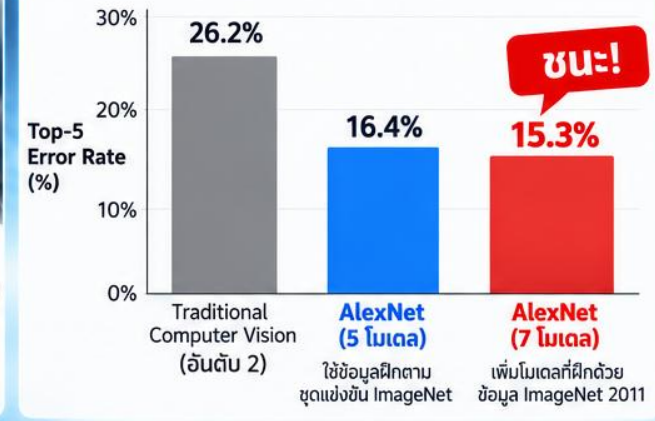




NVIDIA GeForce GTX 580

ชิปที่เด็กเกมเมอร์ใช้เล่นเกม 3 มิติ ถูกนำมาคำนวณเมทริกซ์ของ Deep Learning

ผลการแข่งขัน ImageNet 2012 (Top-5 Error Rate) ยิ่งต่ำยิ่งดี



จาก CPU สู่ GPU : พลังการคำนวณที่เปลี่ยนเกม

CPU (ก่อนปี 2012)

- คำนวณช้า
- เทรนโครงข่ายขนาดใหญ่ไม่ได้
- เหมาะกับงานตรรกะและโปรแกรมทั่วไป



GPU (NVIDIA GeForce GTX 580)

- คำนวณเมทริกซ์ได้เร็วมาก
- เหมาะกับงาน Deep Learning
- ทำให้เทรนโครงข่ายขนาดใหญ่ได้จริง

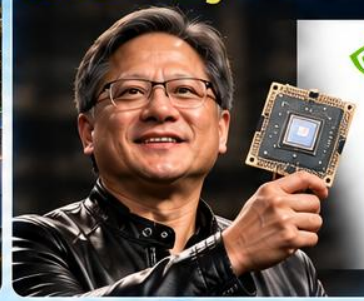


ผลกระทบต่อวงการ AI

- ทั้งซิลิคอนแวลลีย์ตื่นขึ้นมา
- เงินทุนมหาศาลไหลเข้าสู่วงการ
- Google และ Facebook กวาดซื้อตัวนักวิจัยด้าน Deep Learning
- NVIDIA กลายเป็นผู้เล่นสำคัญในอุตสาหกรรม AI



Jensen Huang และการพลิกบทบาทของ NVIDIA



NVIDIA

จากผู้ผลิตชิปสำหรับเกมกราฟิก 3 มิติ สู่หัวใจของการประมวลผล AI ในยุคต่อมา

2010–2011
วิธีดั้งเดิมยังได้ความแม่นยำต่ำ (Top-5 Error ~26%)

2012 (ตุลาคม)
AlexNet ชนะ ImageNet ด้วย GPU (GeForce GTX 580) Top-5 Error 15.3%

2013–2014
Deep Learning ได้รับความสนใจ เงินทุนเริ่มไหลเข้ามา

หลังปี 2012
ยักษ์ใหญ่เทคโนโลยีเริ่มลงทุนและซื้อกวาดนักวิจัย AI จำนวนมาก แต่ในช่วงนั้น AI ยังทำได้แค่แยกแยะภาพหรือแปลประโยคง่าย ๆ

2017 Transformer : จุดเปลี่ยนครั้งสำคัญของ AI ยุคใหม่

จากการอ่านทีละคำ สู่การมองทั้งประโยคพร้อมกัน เข้าใจภาษามนุษย์อย่างแท้จริง

Google นักวิจัย 8 คนของ Google ตีพิมพ์เปเปอร์ประวัติศาสตร์

Attention Is All You Need

Ashish Vaswani* Noam Shazeer* Niki Parmar*
Jakob Uszkoreit* Llion Jones* Aidan N. Gomez † ‡
Lukasz Kaiser* Illia Polosukhin*

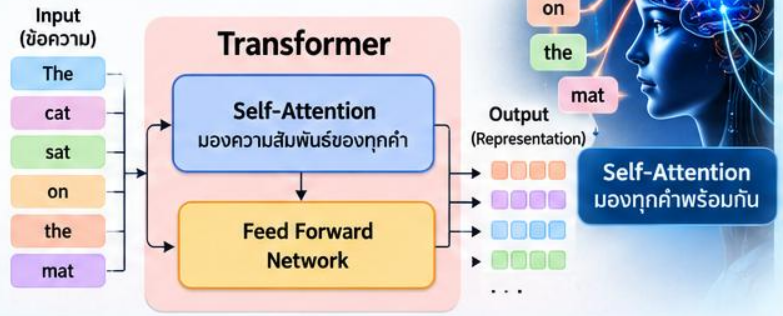
Google Brain
{ashishv, noam, nikip, usz}@google.com

August 31, 2017

"Attention Is All You Need"
แนะนำสถาปัตยกรรมใหม่ที่มีชื่อว่า
Transformer

Transformer คืออะไร ?

Transformer คือสถาปัตยกรรมโครงข่ายประสาทเทียม ที่ใช้กลไก "Self-Attention" ในการมองความสัมพันธ์ของคำทั้งหมดในประโยคพร้อมกัน โดยไม่ต้องอ่านทีละคำ ทำให้เข้าใจบริบทของภาษาได้ดี และประมวลผลได้เร็วขึ้นมาก



ทำไม Transformer ถึงสำคัญ?

- เข้าใจบริบทของภาษาได้ลึกซึ้ง
- จัดการประโยคยาว ๆ ได้ดี
- ประมวลผลแบบขนาน (Parallel) จึงเร็วกว่า RNN/LSTM มาก
- เป็นรากฐานของโมเดลภาษายุคใหม่ เช่น BERT, GPT, T5, LLaMA, Claude และโมเดล Generative AI ทั้งหมดในปัจจุบัน

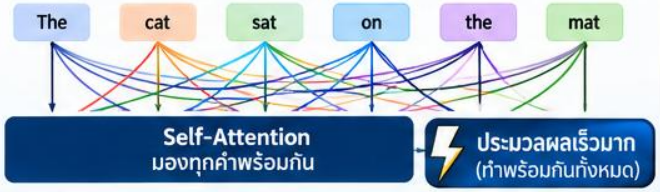
ก่อนมี Transformer : อ่านทีละคำ (RNN/LSTM)

คอมพิวเตอร์อ่านข้อความทีละคำจากซ้ายไปขวาเหมือนคน เมื่ออ่านไปไกล ๆ จะลืมบริบทของคำแรก และประมวลผลได้ช้า



มี Transformer : Self-Attention

สามารถมองทุกคำในประโยคพร้อมกันในเสี้ยววินาที เข้าใจความสัมพันธ์ระหว่างคำได้อย่างแม่นยำ



เช่น ประโยค "The cat chased the mouse because it was hungry."
โมเดลเข้าใจได้ว่า **"it" หมายถึง "cat"** จากบริบทของประโยคทั้งหมด



Google เป็นผู้คิดค้น Transformer ขึ้นมา แต่สตาร์ตอัปเล็ก ๆ ในเวลานั้นอย่าง OpenAI คือบริษัทที่นำมันไปขยายขนาด (Scaling) จนกลายเป็นโมเดลภาษาที่ทรงพลังอย่าง GPT และเปลี่ยนโลก AI ในเวลาต่อมา

2012
Deep Learning
ชนะ ImageNet
ด้วยพลังของ GPU

2017
Attention Is All You Need
เสนอสถาปัตยกรรม Transformer

2018–2020
BERT, GPT-1/2/3
ต่อยอดจาก Transformer

2022–ปัจจุบัน
ChatGPT, Claude, Gemini
Generative AI สู่คนทั่วโลก

จากแนวคิด “ยิ่งใหญ่ขึ้น ยิ่งฉลาดขึ้น” สู่ ChatGPT

The Scaling Hypothesis ที่เปลี่ยน AI จากงานวิจัย สู่เครื่องมือของคนทั้งโลก

1 The Scaling Hypothesis
ยิ่งใหญ่ขึ้น ยิ่งฉลาดขึ้น



Ilya Sutskever
Chief Scientist (Former)
OpenAI

Sam Altman
CEO
OpenAI

“ถ้าเราเอาสถาปัตยกรรม **Transformer** มา**ขยายขนาดให้ใหญ่ขึ้น**เรื่อย ๆ
อัดพลังชิป GPU เข้าไป**นับหมื่นตัว**
และป้อนข้อความทั้งหมดที่มีบนอินเทอร์เน็ต
เข้าไปให้มันทำนายคำถัดไป
(Next-token prediction)
วันหนึ่ง **“เชาวน์ปัญญา”** จะผุดขึ้นมาเอง
โดยไม่ต้องเขียนโค้ดสอนวิธีคิด”

2 ขยายขนาดโมเดล + ข้อมูล + พลังการคำนวณ

Bigger Model (Transformer)



More Data (Internet-scale)



More Compute (Thousands of GPUs)



3 จาก GPT รุ่นแรก ๆ สู่ GPT-3
เริ่มจากโมเดลที่ไม่มีใครสนใจ แต่ค่อย ๆ ใหญ่ขึ้น และเก่งขึ้นเรื่อย ๆ

GPT-1 (2018)	GPT-2 (2019)	GPT-3 (2020)
117M parameters	**1.5B** parameters	**175B** parameters
ยังไม่เป็นที่รู้จักในวงกว้าง	เริ่มสร้างความสนใจ แต่ยังจำกัดการใช้งาน	ความสามารถก้าวกระโดด ทำได้หลากหลายงาน

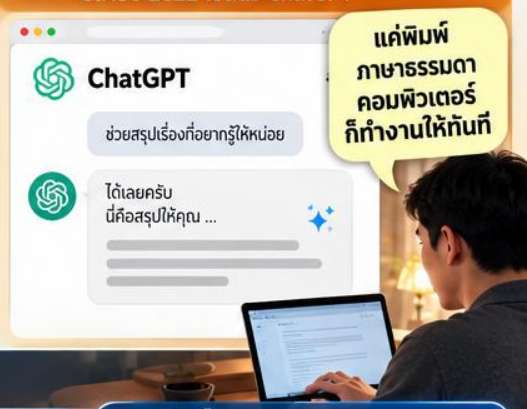
4 RLHF : ทำให้ตอบเหมือนคน มีมารยาท
ปลายปี 2022 ปรับปรุงด้วยการเรียนรู้จากคำติชมของมนุษย์



RLHF
(Reinforcement Learning from Human Feedback)

- ลดการพ่นคำมั่ว
- ตอบถูกต้องและปลอดภัยขึ้น
- เป็นมิตร มีมารยาท
- สอดคล้องกับความต้องการของมนุษย์

5 ChatGPT : อินเทอร์เฟซที่เปลี่ยนทุกอย่าง
ปลายปี 2022 เปิดตัว ChatGPT



6 ผลกระทบ : กำแพงทางเทคนิคพังทลาย สู่ AI สำหรับทุกคน





ผลลัพธ์

- คนธรรมดาเข้าถึง AI ได้
- เกิดการใช้งานในวงกว้าง
- บริษัทเทคโนโลยีเร่งพัฒนาโมเดลใหม่ (Claude, Gemini, ฯลฯ)
- AI กลายเป็นแรงขับเคลื่อนสำคัญของเศรษฐกิจและสังคม

จาก ChatGPT สู่สงคราม AI ระดับโลก

เมื่อประตูถูกเปิดออก ทุกบริษัทใหญ่ก็ต้องเร่งเข้ามาในสมรภูมินี้

ปลายปี 2022

OpenAI เปิดตัว ChatGPT

โมเดลภาษาที่คุยเป็น..ภาษาคนได้ ผ่านอินเทอร์เฟซแชตที่ใช้ง่าย



Sam Altman
CEO, OpenAI

OpenAI



หลังจากนั้น

Dario Amodei แยกตัวออกจาก OpenAI ไปตั้งบริษัท Anthropic เพื่อสร้าง Claude



Dario Amodei
Co-founder & CEO, Anthropic
(Former OpenAI Research Lead)

ANTHROP\C
Claude

มุ่งเน้น AI ที่ปลอดภัย และสอดคล้องกับมนุษย์ (Constitutional AI)

Google ตื่นตระหนก

สั่งควบรวมหน่วยงาน AI แล้วเข็น Gemini ออกมาสู้



Sundar Pichai
CEO, Google

Google

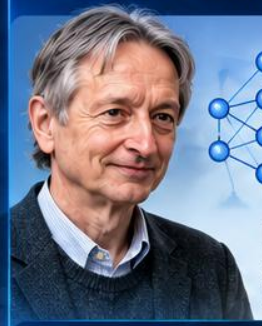
Gemini

รวมทีมวิจัยจาก Google Brain และ DeepMind เพื่อสร้างโมเดล AI รุ่นใหม่

สามรากฐานที่ทำให้เกิด AI ในวันนี้

ไม่ใช่เวทมนตร์ในไม่กี่ปี แต่เกิดจากการบรรจบกันของสามปัจจัยที่ใช้เวลาสร้างเกือบหนึ่งศตวรรษ

1 คณิตศาสตร์โครงข่ายประสาทเทียม ของ Geoffrey Hinton



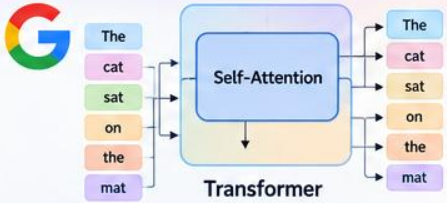


Neural Networks
แนวคิดโครงข่ายประสาทเทียม ให้เครื่องเรียนรู้จากข้อมูล

แนวคิดและงานวิจัยยาวนานหลายสิบปี

1980s – 2010s

2 สถาปัตยกรรม Transformer ของที่มวิจัย Google



กุญแจที่ทำให้เข้าใจความสัมพันธ์ของข้อมูล ได้ทั้งประโยคพร้อมกัน

งานวิจัยและตีพิมพ์ในปี 2017

(Attention Is All You Need)

3 ความบ้าบิ่นในการเติมพัน ด้วยเงินและพลังประมวลผลมหาศาล ของ OpenAI

OpenAI



- ขยายโมเดลให้ใหญ่ขึ้น (Scaling)
- ใช้ GPU นับหมื่นตัว
- ป้อนข้อมูลทั้งหมดบนอินเทอร์เน็ต
- ใช้ Next-token prediction และ RLHF ให้ตอบเหมือนคน

จาก GPT-1 → GPT-3 → ChatGPT

2020 – 2022

AI ในวันนี้



ทั้งสามปัจจัยมาบรรจบกัน จึงเกิด **AI ที่เปลี่ยนโลกอย่างแท้จริง** และยังคงเดินหน้าต่อไป



AI หรือ ปัญญาประดิษฐ์ (Artificial Intelligence)

เป็นเทคโนโลยีการสร้างความสามารถให้แก่เครื่องจักรและคอมพิวเตอร์ ด้วยอัลกอริทึมและกลุ่มเครื่องมือทางสถิติ เพื่อสร้าง**ซอฟต์แวร์ทรงปัญญา** ที่สามารถ**เลียนแบบความสามารถของมนุษย์ที่ซับซ้อนได้**
เช่น จดจำ แยกแยะ ให้เหตุผล ตัดสินใจ คาดการณ์ สื่อสารกับมนุษย์ ในบางกรณีอาจไปถึงขั้นเรียนรู้ได้ด้วยตนเอง

ข้อมูล (Data)

อัลกอริทึมและเครื่องมือทางสถิติ (Algorithms & Statistical Tools)

AI ซอฟต์แวร์ทรงปัญญา (Intelligent Software)

ความสามารถ (มนุษย์ที่ซับซ้อนได้)
- จดจำ
- แยกแยะ
- ให้เหตุผล
- ตัดสินใจ
- คาดการณ์
- สื่อสารกับมนุษย์
- เรียนรู้ได้ด้วยตนเอง

ตัวอย่างความสามารถของ AI

จดจำ (Recognition)


จดจำวัตถุ ใบหน้า เสียง หรือรูปแบบต่าง ๆ

แยกแยะ (Classification)


แยกแยะประเภทหรือจัดหมวดหมู่ข้อมูล

ให้เหตุผล (Reasoning)


วิเคราะห์ความสัมพันธ์และให้เหตุผลเหมือนมนุษย์

ตัดสินใจ (Decision Making)

ตัดสินใจจากข้อมูลในสถานการณ์ที่ซับซ้อน

คาดการณ์ (Prediction)


คาดการณ์แนวโน้ม เช่น ยอดขาย สภาพอากาศ หรือความเสี่ยง

สื่อสารกับมนุษย์ (Natural Language)


เข้าใจและสื่อสารด้วยภาษาบมนุษย์

เรียนรู้ได้ด้วยตนเอง (Self Learning)

ปรับปรุงความสามารถจากประสบการณ์และข้อมูลใหม่ ๆ โดยไม่ต้องเขียนกฎเกณฑ์ทุกอย่าง

ตัวอย่างการใช้งาน AI ในชีวิตจริง

- **การแพทย์** ช่วยวินิจฉัยโรคจากภาพถ่าย
- **การเดินทาง** รถยนต์ขับเคลื่อนอัตโนมัติ
- **ธุรกิจ** แนะนำสินค้าเฉพาะบุคคล
- **การศึกษา** ติวเตอร์อัจฉริยะ
- **ความปลอดภัย** ตรวจจับความผิดปกติ
- ••• **อีกมากมาย**

การประยุกต์ใช้ AI กับ IoT

AI เพิ่มความสามารถให้ IoT จากการตรวจวัดและควบคุม ไปสู่ ระบบที่วิเคราะห์ ตีความ และตัดสินใจได้



ประเภทของการประยุกต์ใช้ AI กับ IoT

1. Natural Language AI Control

คือ การแปลงภาษาธรรมชาติ ให้เป็นคำสั่งเพื่อควบคุมอุปกรณ์ หรือรับค่าจากอุปกรณ์
เมื่อ User พิมพ์หรือพูดภาษาธรมชาติ แล้ว LLM จะเป็นตัวหลักในการตีความ ภาษาธรรมชาติ ให้เป็นคำสั่งเพื่อควบคุมอุปกรณ์ หรือรับค่าจากอุปกรณ์

ตัวอย่าง : พิมพ์ “เปิดไฟ” หรือ “อ่านอุณหภูมิ” แล้วให้ AI สั่ง Arduino ผ่าน MQTT

2. AI Data Analytics & Decision Support

คือ การรับข้อมูลจากอุปกรณ์ เพื่อนำไปวิเคราะห์ และสนับสนุนการตัดสินใจ โดย AI จะ รวบรวมและเตรียมข้อมูลจากอุปกรณ์ , วิเคราะห์ข้อมูลจากอุปกรณ์ , เพื่อหาความผิดปกติ (Anomaly Detection) หรือทำการ Classification หรือทำการ พยากรณ์ หรือ แนะนำการตัดสินใจ รวมถึงการใช้ LLM สรุปผลและอธิบายความผิดปกติ

ตัวอย่าง : วิเคราะห์อุณหภูมิและความชื้น พร้อมตรวจจับค่าผิดปกติ

ประเภทของการประยุกต์ใช้ AI กับ IoT

3. Edge AI / TinyML

คือ การนำ AI ไปประมวลผลบนอุปกรณ์ โดยจะนำโมเดลที่ฝึกแล้วซึ่งติดตั้งไว้บนอุปกรณ์ เมื่ออุปกรณ์รับข้อมูลจากเซนเซอร์ต่าง ๆ มาเข้าโมเดล จะสามารถจำแนกข้อมูลได้อย่าง Realtime หรือทำนายค่าได้ทันที โดยไม่ต้องส่งข้อมูลไปประมวลผลที่ Cloud ทุกครั้ง แต่อุปกรณ์อาจจะมีข้อจำกัด CPU, RAM , พลังงาน , Latency

ตัวอย่าง : จำแนกสถานะจาก Sensor โดยไม่ต้องเรียก Cloud AI

4. AI Vision & Sensor Fusion

คือ การใช้ AI วิเคราะห์ ภาพ เสียง รวมข้อมูลจากเซนเซอร์หลายชนิด เพื่อให้ระบบเข้าใจสถานการณ์และ ตอบสนองตามสถานการณ์นั้น ๆ เช่น การใช้ Computer Vision ในการวิเคราะห์ภาพเพื่อควบคุมอุปกรณ์ โดยรับภาพจาก Raspberry Pi Camera และใช้ OpenCV และ Object Detection ในการตรวจจับ คน รถ หรือวัตถุ และการแปลงผลการตรวจจับ เพื่อให้ระบบเข้าใจสถานการณ์ และทำการสรุปผล หรือ อธิบายความผิดปกติ

Natural Language AI Control

คือ การแปลงภาษาธรรมชาติ ให้เป็นคำสั่งเพื่อควบคุมอุปกรณ์ หรือรับค่าจากอุปกรณ์

โดย user พิมพ์หรือพูดภาษาธรรมชาติ แล้ว LLM จะเป็นตัวหลักในการตีความภาษาธรรมชาติ ให้เป็นคำสั่งเพื่อควบคุมอุปกรณ์ หรือรับค่าจากอุปกรณ์

User

พิมพ์หรือพูด
ภาษาธรรมชาติ



Text Voice

ข้อความ

เสียง

LLM

ตีความภาษาธรรมชาติ
และวิเคราะห์เจตนา

- เข้าใจความหมาย
- ระบุเจตนา (Intent)
- แยกข้อมูลที่เกี่ยวข้อง
- เลือกคำสั่ง/ฟังก์ชันที่เหมาะสม

เจตนา : เปิดไฟ
อุปกรณ์ : LED ห้องทำงาน
คำสั่ง : set_led(on)

คำสั่ง
ที่ตีความได้

```
{
  "action": "set_led",
  "device": "led_room",
  "value": "on"
}
```

ข้อมูลจากอุปกรณ์

```
{
  "temperature": 28.5,
  "unit": "C"
}
```

MCP Server / Function Calling

เรียกใช้ฟังก์ชัน
และเชื่อมต่ออุปกรณ์

- set_led(device, state)
- get_temperature()
- get_sensor_data()
- ...

MQTT / API / GPIO

MQTT API GPIO

ส่งคำสั่ง / รับข้อมูล

ควบคุม
อุปกรณ์

ข้อมูลจาก
อุปกรณ์

IoT Devices

อุปกรณ์ในระบบ IoT



ตัวอย่างการใช้งาน

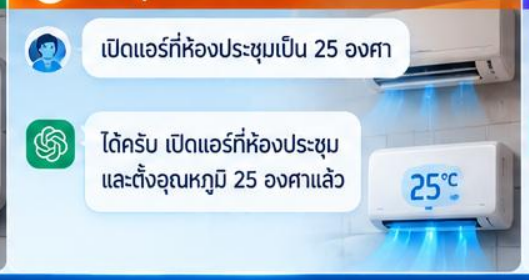
1 สั่งเปิดไฟ



2 สอบถามค่าเซนเซอร์



3 ควบคุมเครื่องปรับอากาศ



4 รับข้อมูลหลายค่า



AI Data Analytics & Decision Support

คือ การรับข้อมูลจากอุปกรณ์ เพื่อนำไปวิเคราะห์ และสนับสนุนการตัดสินใจ

โดย AI จะรวบรวมและเตรียมข้อมูลจากอุปกรณ์ วิเคราะห์ข้อมูลจากอุปกรณ์ เพื่อหาความผิดปกติ ทำการ Classification พยากรณ์ แนะนำการตัดสินใจ รวมถึงการใช้ LLM สรุปผลและอธิบายความผิดปกติ

1 อุปกรณ์และแหล่งข้อมูล (IoT Devices)

- เซ็นเซอร์อุณหภูมิ/ความชื้น
- เครื่องจักรในโรงงาน
- กล้อง/ภาพ
- มิเตอร์พลังงาน
- เซ็นเซอร์ระดับน้ำ
- เซ็นเซอร์การเกษตร
- ... อุปกรณ์อื่น ๆ

2 รวบรวมและเตรียมข้อมูล (Data Ingestion & Preparation)

MQTT · HTTP · LoRa · Wi-Fi · Ethernet · 4G/5G

- รวบรวมข้อมูลจากอุปกรณ์
- จัดเก็บในฐานข้อมูล
- ทำความสะอาดข้อมูล
- แปลงรูปแบบและรวมข้อมูลจากหลายแหล่ง

Time Series Database · Data Lake · Data Warehouse

3 วิเคราะห์ข้อมูลด้วย AI (AI Analytics)

Anomaly Detection
ตรวจหาความผิดปกติ



Classification
จำแนกประเภท

ปกติ · เฝ้าระวัง · ผิดปกติ

Prediction / Forecasting
พยากรณ์แนวโน้ม



Recommendation
แนะนำการตัดสินใจ

- แนะนำการปรับตั้งค่า
- แนะนำการบำรุงรักษา
- แนะนำการใช้ทรัพยากร

4 LLM สรุปผลและอธิบาย (Insight & Explanation)

LLM

สรุปผลการวิเคราะห์ อธิบายสาเหตุของความผิดปกติ และให้ข้อเสนอแนะด้วยภาษาที่เข้าใจง่าย

เกิดอะไรขึ้นกับเครื่องจักร A ในวันนี้ ?

วันนี้เครื่องจักร A มีอุณหภูมิสูงกว่าปกติ ตั้งแต่เวลา 10:15 น. ซึ่งอาจเกิดจากการทำงานหนักต่อเนื่อง แนะนำให้ตรวจสอบระบบระบายความร้อน และวางแผนหยุดเครื่องเพื่อบำรุงรักษาภายใน 1 สัปดาห์

5 สนับสนุนการตัดสินใจ (Decision Support)



ตัวอย่างการใช้งานจริง

โรงงานอัจฉริยะ (Smart Factory)



- ตรวจจับความผิดปกติเครื่องจักร
- พยากรณ์การซ่อมบำรุง (Predictive Maintenance)
- แนะนำการปรับการผลิต

อาคารอัจฉริยะ (Smart Building)



- วิเคราะห์การใช้พลังงาน
- ตรวจหาความผิดปกติระบบ HVAC
- แนะนำการควบคุมให้ประหยัดพลังงาน

เกษตรอัจฉริยะ (Smart Agriculture)



- พยากรณ์ผลผลิต
- ตรวจจับโรคพืชจากภาพ
- แนะนำการให้น้ำ/ปุ๋ย

เมืองอัจฉริยะ (Smart City)



- วิเคราะห์การจราจร
- ตรวจจับเหตุผิดปกติ
- คาดการณ์ความหนาแน่นและแนะนำการจัดการ

พลังงานและสาธารณูปโภค (Smart Energy)



- พยากรณ์การใช้พลังงาน
- ตรวจจับความผิดปกติของระบบไฟฟ้า
- แนะนำการบริหารจัดการโหลด

สาธารณสุข/สุขภาพ (Smart Health)



- ติดตามสุขภาพผู้สูงอายุ
- ตรวจจับความผิดปกติ
- แจ้งเตือนและแนะนำการดูแล

Edge AI / TinyML

คือ การนำ AI ไปประมวลผลบนอุปกรณ์ โดยใช้โมเดลที่ฝึกแล้วซึ่งติดตั้งไว้บนอุปกรณ์

เมื่ออุปกรณ์รับข้อมูลจากเซนเซอร์ต่าง ๆ มาเข้าโมเดล จะสามารถจำแนกข้อมูลได้อย่าง Real-time หรือทำนายค่าได้ทันที โดยไม่ต้องส่งข้อมูลไปประมวลผลที่ Cloud ทุกครั้ง

1 รับข้อมูลจากเซนเซอร์ (Sensor Input)

- ภาพ/วีดีโอ
- เสียง
- อุณหภูมิ
- การสั่นสะเทือน
- ความชื้น
- คุณภาพอากาศ (CO_2)
- ตำแหน่ง
- เซนเซอร์อื่น ๆ

ข้อมูลจากเซนเซอร์

2 ประมวลผลบนอุปกรณ์ (Edge Device + TinyML)



โมเดลที่ฝึกแล้วติดตั้งในอุปกรณ์

Raspberry Pi · ESP32 · Arduino · MCU

TinyML Model

ข้อมูลเข้า (Sensor Data) → ผลลัพธ์ (Classification / Prediction)

3 ผลลัพธ์แบบ Real-time บนอุปกรณ์



จำแนกภาพได้ทันที

เสียงผิดปกติ (Anomaly)

ตรวจจับความผิดปกติแบบทันที

32.5 °C

ทำนายค่าล่วงหน้า

4 สั่งงานอุปกรณ์ / แสดงผล

- แจ้งเตือน
- ควบคุมอุปกรณ์
- ทำงานอัตโนมัติ
- แสดงผลบนหน้าจอ (32.5 °C, 60%)

ส่งข้อมูลไป Cloud (เฉพาะข้อมูลสำคัญ/สรุปผล)

เก็บข้อมูลระยะยาว เพื่อวิเคราะห์เพิ่มเติม

ตัวอย่างการใช้งาน Edge AI / TinyML

Smart Camera (จำแนกภาพ)



- ตรวจจับคน/ยานพาหนะ
- ตรวจหาสิ่งผิดปกติ
- ความปลอดภัย

Predictive Maintenance



- วิเคราะห์การสั่นสะเทือน
- ตรวจจับความผิดปกติของเครื่องจักร
- แจ้งเตือนก่อนเกิดความเสียหาย

Smart Agriculture



- พยากรณ์ผลผลิต
- ควบคุมการให้น้ำ/ปุ๋ยอัตโนมัติ
- ประหยัดพลังงาน

Smart Home / IoT



- ตรวจคุณภาพอากาศ
- ควบคุมอุปกรณ์ในบ้าน
- ทำงานแบบออฟไลน์ได้

ข้อจำกัดของอุปกรณ์ Edge AI

CPU	ประสิทธิภาพการประมวลผลจำกัด
RAM	หน่วยความจำจำกัด
พลังงาน	ใช้พลังงานต่ำ (โดยเฉพาะอุปกรณ์พกพา)
Latency	ต้องการการตอบสนองแบบ Real-time
ขนาดโมเดล	ต้องใช้โมเดลขนาดเล็ก (TinyML)
การอัปเดต	การอัปเดตโมเดลและการบำรุงรักษา

AI Vision & Sensor Fusion

คือ การใช้ AI วิเคราะห์ภาพ เสียง รวมข้อมูลจากเซนเซอร์หลายชนิด เพื่อให้ระบบเข้าใจสถานการณ์ และตอบสนองตามสถานการณ์นั้น ๆ

ตัวอย่างเช่น การใช้ Computer Vision ในการวิเคราะห์ภาพเพื่อควบคุมอุปกรณ์ โดยรับภาพจาก Raspberry Pi Camera และใช้ OpenCV และ Object Detection ในการตรวจจับ คน รถ หรือวัตถุ แล้วรวมข้อมูลจากเซนเซอร์อื่น ๆ เพื่อให้ระบบเข้าใจสถานการณ์ ทำการสรุปผล หรืออธิบายความผิดปกติ

1 รับข้อมูลจากหลายเซนเซอร์ (Multi-Modal Sensors)

- กล้อง (Raspberry Pi Camera) ภาพ/วิดีโอ
- ไมโครโฟน เสียง
- เซนเซอร์อุณหภูมิ/ความชื้น
- เซนเซอร์ระยะทาง (Ultrasonic)
- เซนเซอร์การเคลื่อนไหว (PIR)
- ตำแหน่ง (GPS)
- ... เซนเซอร์อื่น ๆ

2 ประมวลผลด้วย AI (Computer Vision + Sensor Fusion)

Raspberry Pi · OpenCV · YOLO (Object Detection)



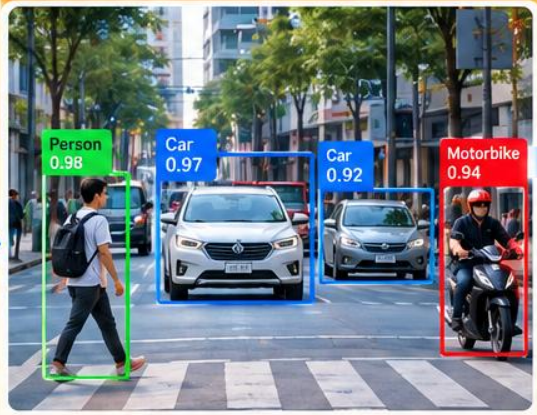
การประมวลผลภาพ (Image Processing)

ตรวจจับและจำแนกวัตถุ (Object Detection & Classification)

- คน (Person)
- รถ (Vehicle)
- วัตถุ (Object)

รวมข้อมูลจากเซนเซอร์หลายชนิด (Sensor Fusion)

3 ผลการวิเคราะห์ (Understanding the Situation)



ข้อมูลจากเซนเซอร์อื่น ๆ

- 32.5 °C
- 65 %RH
- เสียง: 70 dB
- พิกัด: 13.729, 100.754

การตีความเสถานการณ์

- พบคนก้าลังข้ามถนน
- พบรถยนต์ 2 คัน
- พบรถจักรยานยนต์ 1 คัน
- เสียงดังผิดปกติ (70 dB)
- อุณหภูมิสูงขึ้น
- ระยะห่างวัตถุใกล้ (1.2 m)

4 การตอบสนองของระบบ (Action & Automation)

- สิ่งเปิด/ปิด ไม้กั้น
- แจ้งเตือน (เสียง/ไฟ)
- บันทึกเหตุการณ์
- ส่งข้อมูลไปยัง Dashboard
- ควบคุมอุปกรณ์อื่น ๆ

5 สรุปผลและอธิบายด้วย AI (LLM)

วันนี้เวลา 14:23 น. ตรวจพบว่ามีคนและรถยนต์บนถนน ปริมาณการจราจรปกติ แต่พบว่าระดับเสียงสูงกว่าปกติ (70 dB) ซึ่งอาจเกิดจากการก่อสร้างใกล้เคียง แนะนำให้ตรวจสอบพื้นที่เพิ่มเติม

ตัวอย่างการใช้งานจริง

Smart Parking



ตรวจจับรถ และควบคุมไม้กั้น

Smart Security



ตรวจจับคนบุกรุก แจ้งเตือนทันที

Smart Factory



ตรวจจับการสวมอุปกรณ์ป้องกัน

Smart Traffic



นับจำนวนรถ วิเคราะห์การจราจร

Smart Home / Building



ตรวจจับคน ควบคุมไฟ/แอร์

Environment Monitoring



ตรวจจับควัน/ไฟป่า แจ้งเตือน

สถาปัตยกรรมการเชื่อมต่อ AI กับ IoT

เป็นการแสดงถึงว่า เราจะใช้ AI ประมวลผลได้ที่ไหน รับข้อมูลจากไหน และส่งคำสั่งไปยังอุปกรณ์อย่างไร แบ่งได้ดังนี้

- Cloud AI Integration : เชื่อม IoT กับ AI Model ซึ่งทำงานอยู่บน Cloud ผ่านทาง REST API และ HTTPS จะต้องมี API Key และ Authentication เช่น การส่งข้อมูลไปให้ AI วิเคราะห์
- MQTT-Based AIoT Integration : ใช้โปรโตคอล MQTT เป็นช่องทางในการรับส่งข้อมูลและคำสั่ง ระหว่างอุปกรณ์ IoT กับ AI Model โดยใช้การ Publish/Subscribe และเชื่อม AI Model เข้ากับ MQTT Broker เช่น วิเคราะห์ข้อความ แล้ว publish ไปปิด-เปิดอุปกรณ์
- Edge AI and Local Integration : ทำการประมวลผล AI บนอุปกรณ์นั้น ๆ หรือทำการ Run model บน Server ขององค์กร สามารถทำงานแบบ Offline ได้ และช่วยลด Latency เช่น วิเคราะห์ภาพจากกล้อง แล้วสั่งให้อุปกรณ์ทำงาน

สถาปัตยกรรมการเชื่อมต่อ AI กับ IoT

- LLM Tool Calling and Function Calling : ให้ LLM เข้าใจคำสั่งภาษาธรรมชาติ แล้วสามารถเลือกฟังก์ชันที่สร้างไว้ เช่น ฟังก์ชันเพื่อควบคุมการทำงานของอุปกรณ์ (LLM ไม่ได้ต่ออุปกรณ์โดยตรง แต่จะเรียกผ่านฟังก์ชันที่ตั้งไว้) โดยในฟังก์ชัน จะมีการกำหนด Function schema และ Parameters เช่น สร้างฟังก์ชัน set_led() แล้ว LLM จะวิเคราะห์เจตนาของผู้ใช้ แล้วเลือก ฟังก์ชันที่ตรงกับงาน

- Model Context Protocol (MCP) : คือ มาตรฐานที่ช่วยให้ AI สามารถเชื่อมต่อกับเครื่องมือ หรือ ข้อมูลภายนอก เช่น กล้อง ฐานข้อมูล ไฟล์ หรือระบบ IoT หรือ เชื่อมต่อกับบริการต่าง ๆ ภายนอกได้อย่างเป็นระบบ แบ่งเป็น MCP Client และ MCP Server โดยใน MCP Server จะให้บริการ Tools, Resources และ Prompts และมีช่องทางที่ MCP Client ใช้สื่อสารกับ MCP Server อยู่ 2 รูปแบบ คือ stdio และ Streamable HTTP เช่น สร้าง MCP Server บน Raspberry Pi เพื่อให้ AI อ่านสถานะกล้องและสั่งเปิดไฟผ่าน MQTT

- AIoT Gateway and System Integration : คือ การใช้อุปกรณ์ที่เป็นการเชื่อม AI, IoT, MQTT, API และระบบต่าง ๆ เข้าด้วยกันเป็นระบบเดียว จะมีลักษณะเป็นทั้ง API Gateway และ Protocol Bridging มีระบบ Logging, Monitoring และ Error Handling เป็นต้น

AIoT Connectivity & Integration Architecture

สถาปัตยกรรมและเทคโนโลยีที่ใช้เชื่อม AI กับ IoT

1 Cloud AI Integration

- ส่งข้อมูลจากอุปกรณ์ไปยัง Cloud
- AI วิเคราะห์/พยากรณ์/สรุปผล
- ส่งคำสั่งกลับไปยังอุปกรณ์

2 MQTT-Based AIoT Integration

รับส่งข้อมูลและคำสั่งผ่าน MQTT

Publish Sensor Data — Subscribe Commands

- อุปกรณ์ส่งข้อมูล (Publish)
- AI หรือระบบอื่นรับข้อมูล (Subscribe)
- ส่งคำสั่งกลับไปยังอุปกรณ์ผ่าน MQTT

3 Edge AI and Local Integration

ประมวลผล AI บนอุปกรณ์หรือเซิร์ฟเวอร์ภายใน

- ประมวลผล AI บนอุปกรณ์ (Edge)
- ตอบสนองแบบ Real-time
- ไม่ต้องส่งข้อมูลไป Cloud ทุกครั้ง
- เหมาะกับงานที่ต้องการ Latency ต่ำ

4 LLM Tool Calling and Function Calling

ให้ LLM เลือกเรียกฟังก์ชันเพื่อควบคุมระบบ

- get_sensor_data()
- control_device()
- analyze_image()
- send_notification()

- ผู้ใช้สั่งงานด้วยภาษาธรรมชาติ
- LLM เลือกเรียกฟังก์ชัน (Tool)
- เชื่อมต่อกับ IoT/ระบบต่าง ๆ

5 Model Context Protocol (MCP)

มาตรฐานเชื่อม AI กับ Tools และบริการภายนอก

MCP → Tools, Resources, Prompts, External Services

- มาตรฐานเชื่อมต่อ AI กับ Tools
- รองรับการทำงานข้ามระบบ
- ใช้งานร่วมกับ LLM และ Agent

6 AIoT Gateway and System Integration

เชื่อม AI, IoT และระบบต่าง ๆ เข้าด้วยกัน

AIoT Gateway → IoT Devices, AI / LLM, MQTT Broker, Database, Business Systems (ERP, SCADA ฯลฯ)

- รวบรวมและจัดการการเชื่อมต่อ
- แปลงโปรโตคอล/ข้อมูล
- เชื่อมต่อ AI กับระบบองค์กร

IoT Devices (Sensors/Actuators)

- เซ็นเซอร์อุณหภูมิ/ความชื้น
- กล้อง/ภาพ
- ไมโครโฟน/เสียง
- เครื่องจักรในโรงงาน
- มิเตอร์พลังงาน
- เซ็นเซอร์การเกษตร
- อื่น ๆ

Sensor Data (MQTT/HTTP/CoAP) →

← Control Commands (MQTT/HTTP)

Edge AI / Local Integration

- ประมวลผล AI บนอุปกรณ์
- เช่น Object Detection
- Classification
- Control แบบ Real-time

Raspberry Pi, Jetson, Industrial PC

Data (MQTT/HTTP) →

← Commands

Cloud AI (LLM / AI Platform)

OpenAI, Gemini, aws, Azure

Sensor Data →

← Analysis Result / Commands

- วิเคราะห์ข้อมูล
- พยากรณ์
- สรุปผล
- ปรับแต่งโมเดล
- เก็บข้อมูลระยะยาว

MQTT Broker

LLM + Tool Calling (ผ่าน MCP)

MCP

- รับคำสั่งภาษาธรรมชาติ
- เลือกเรียกฟังก์ชัน (Tool)
- ควบคุมอุปกรณ์ / ดึงข้อมูล
- ใช้งานร่วมกับระบบต่าง ๆ

User (Natural Language)

เปิดไฟที่ห้องประชุมและรายงานอุณหภูมิให้หน่อย

Data Storage & Integration

Time Series DB, Database, Data Lake, API / Webhook

Business Systems

Dashboard, ERP, SCADA, Notification

ตัวอย่างการทำงาน

1 รับข้อมูลจากอุปกรณ์



กล้องส่งภาพรถเข้าผ่าน MQTT

2 ประมวลผล AI



Edge AI ตรวจจับรถด้วย Computer Vision

3 วิเคราะห์/สรุปผล

AI/LLM วิเคราะห์ข้อมูล สรุปจำนวนรถ และตรวจพบความผิดปกติ

4 ส่งคำสั่งไปยังอุปกรณ์



สั่งเปิดไม้กั้นผ่าน MQTT

5 แสดงผล/แจ้งเตือน



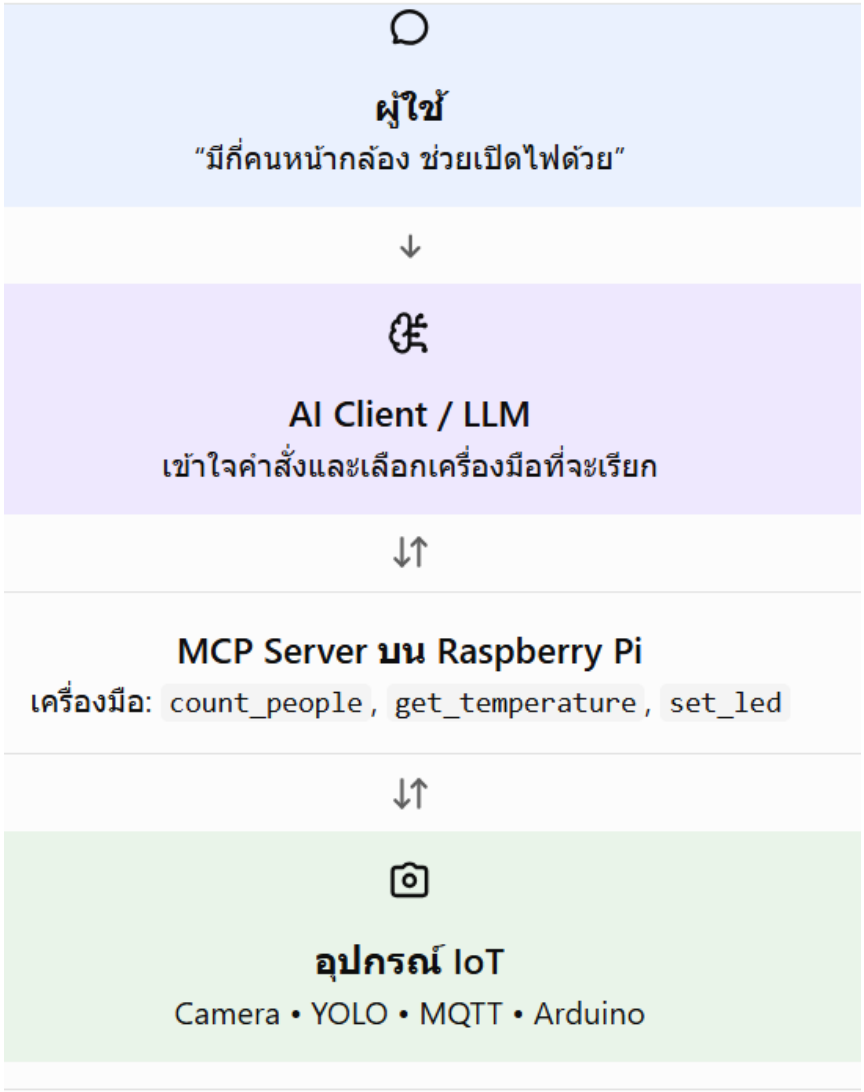
แสดงผลบน Dashboard และส่งการแจ้งเตือน

Model Context Protocol (MCP)

- MCP (Model Context Protocol) คือ มาตรฐานที่ช่วยให้ AI สามารถเชื่อมต่อกับเครื่องมือ หรือ ข้อมูลภายนอกได้ เช่น กล้อง ฐานข้อมูล ไฟล์ หรือระบบ IoT หรือ บริการภายนอกได้อย่างเป็นระบบ

- ข้อดี
  - ลดการยึดติดกับ AI Model ตัวใดตัวหนึ่ง
  - สามารถเพิ่ม Tools ได้โดยง่าย
  - มีโครงสร้างการเชื่อมต่อที่เป็นมาตรฐาน

Model Context Protocol (MCP)



MCP Server ไม่ใช่ AI Model แต่เป็นส่วนที่เปิดให้ AI สามารถเรียกใช้ Tool ได้ตามสิทธิ์ที่กำหนด

เช่น MCP Server เปิด Tool ให้ใช้งาน 3 ตัว

count_people	ตรวจภาพจากกล้องและนับจำนวนคน
get_temperature	อ่านอุณหภูมิผ่าน MQTT
set_led	ส่งคำสั่งเปิดหรือปิด LED ผ่าน MQTT

MCP Client & MCP Server

- MCP Client คือ ฝั่งที่อยู่ใน AI Application ทำหน้าที่เชื่อมต่อกับ MCP Server และส่งคำขอเพื่อค้นหาหรือเรียกใช้ Tool

- MCP Server คือ ฝั่งที่ประกาศเปิดเผย Tool และข้อมูลให้แก่ Client เพื่อเรียกใช้งาน โดยภายใน Server อาจจะเรียก Python Function, MQTT, Database หรือ API ของอุปกรณ์ IoT และดำเนินการตามคำขอของ Client

องค์ประกอบหลัก ของ MCP Server

- **Tools คือ สิ่งที่ AI เรียกสั่งให้ทำงานได้ , เครื่องมือสำหรับลงมือทำ**
  - เป็นคำสั่ง ที่เปิดให้โมเดล AI ตัดสินใจเลือก "เรียกใช้" ได้ด้วยตัวเองตามบริบทของการสนทนา
  - AI จะเป็นคนคิดและเลือกเองว่า ควรจะใช้ Tool ตัวไหน ส่งพารามิเตอร์อะไร เพื่อกระทำบางอย่าง (Action) เช่น
    - set_led(action) เมื่อรับบริบทสนทนา เป็น เปิดหรือปิดไฟ
    - get_temperature() เมื่อรับบริบทสนทนา เป็น อ่านอุณหภูมิ
    - count_people() เมื่อรับบริบทสนทนา เป็น นับจำนวนคนจากภาพ
    - capture_image() เมื่อรับบริบทสนทนา เป็น ถ่ายภาพจากกล้อง

- **Resources คือ คลังข้อมูล /แหล่งข้อมูล ที่เปิดให้ AI อ่านได้**
  - คือ ข้อมูลหรือเนื้อหาจากระบบภายนอกที่ MCP Server เตรียมไว้ให้ AI ในการเข้าถึงและอ่านได้ โดยมี URI ระบุตัวตน (คล้ายกับการแปะลิงก์หรือเปิดไฟล์ให้ AI อ่าน)
  - ฝั่งแอปพลิเคชันหรือตัว AI จะเป็นฝ่ายดึงข้อมูลเหล่านี้เข้ามาประกอบบริบท (Context) เพื่อใช้ในการตอบคำถาม ไม่ใช่การสั่งให้ทำแอ็กชัน
  - ข้อมูลอาจประกอบด้วยอุณหภูมิล่าสุด รายชื่ออุปกรณ์ หรือสถานะกล้อง โดย Resources มักเหมาะกับข้อมูลที่ต้องการให้อ่าน มากกว่าการสั่งงานอุปกรณ์ หรือ ไฟล์เอกสาร PDF, เนื้อหาจากฐานข้อมูล (Database), โค้ดโปรเจกต์ หรือ Log ไฟล์ต่างๆ

องค์ประกอบหลัก ของ MCP Server

- **Prompts คือ แม่แบบคำสั่งสำหรับ AI หรือแม่แบบการสนทนา**

เป็น ชุดคำสั่งสำเร็จรูปที่เตรียมไว้ล่วงหน้า (Pre-defined prompts) เพื่อให้ผู้ใช้งานหรือระบบเรียกใช้งานได้ง่าย เปรียบเสมือนปุ่มลัด (Shortcut) หรือ Prompt ที่ถูกเซ็ตโครงสร้างไว้อย่างดีแล้ว เพื่อให้ AI เริ่มต้นทำงานตามรูปแบบที่กำหนดไว้ได้อย่างมีประสิทธิภาพ โดยผู้ใช้เป็นคนเลือกกดใช้งาน

- ตัวอย่างชื่อ Prompt
  - analyze_room : วิเคราะห์สถานะห้อง
  - daily_energy_report : สรุปการใช้พลังงาน
  - inspect_camera : ตรวจสอบสิ่งที่กล้องตรวจพบ
  - *"สรุปรายงานการประชุมประจำสัปดาห์"* ที่กำหนดรูปแบบ Output ไว้ชัดเจน

ช่องทางการสื่อสารระหว่าง MCP Client กับ MCP Server

- **stdio :** สื่อสารผ่าน Standard Input/Output
  - MCP Client เปิดโปรแกรม MCP Server เป็น Process แล้วส่งข้อความผ่าน stdin และรับผลผ่าน stdout ปกติจะใช้ในกรณีที่อยู่ที่เครื่องเดียวกัน

- **Streamable HTTP :** สื่อสารผ่าน HTTP
  - MCP Client ส่งคำขอไปยัง HTTP Endpoint ของ MCP Server เช่น /mcp และรับผลกลับผ่าน HTTP โดยรองรับการสตรีมผลลัพธ์ได้ตามความเหมาะสม ปกติจะใช้งานผ่านเครือข่าย (Network accessibility) และรองรับผู้ใช้งานหลายคนพร้อมกัน (Multi-user)

Authentication และ Authorization

- Authentication คือ การพิสูจน์ตัวตน ในการเข้าเชื่อมต่อ MCP Server เช่น ผู้ใช้หรือ AI Client ที่ได้รับอนุญาต

- Authorization คือ การกำหนดสิทธิ์ว่าผู้เชื่อมต่อนั้น สามารถทำอะไรได้บ้าง เช่น อ่านอุณหภูมิได้ แต่ไม่มีสิทธิ์สั่งปิดระบบ

MCP คืออะไร?
(Model Context Protocol)



MCP คือ “ภาษากลาง”
ที่ทำให้ AI (เช่น ChatGPT)
สามารถคุยกับโปรแกรมหรืออุปกรณ์
ภายนอกได้อย่างเป็นระบบ
และปลอดภัย เหมือนมีสะพานเชื่อมกัน
ให้เข้าใจกันได้ครับ

ลองนึกภาพแบบง่าย ๆ



MCP ก็เหมือน “ล่าม” ที่ช่วยให้ AI เข้าใจคำสั่งของเรา
และส่งคำสั่งไปยังโปรแกรมหรืออุปกรณ์ต่าง ๆ ได้

MCP ทำงานอย่างไร?

1 คุณสั่งงานด้วยภาษาธรรมดา



ChatGPT
(หรือแอปที่รองรับ MCP)

2 แปลงเป็นคำสั่ง MCP

```
{
  "tool": "set_led",
  "color": "red"
}
```

MCP จะเปลี่ยนคำสั่งของเรา
ให้เป็นรูปแบบที่โปรแกรมเข้าใจได้

3 โปรแกรม (MCP Server) ทำงานตามคำสั่ง



MCP Server
(เช่น โปรแกรม Python)

MCP Server จะรับคำสั่ง
แล้วสั่งงานไปยังอุปกรณ์ต่าง ๆ

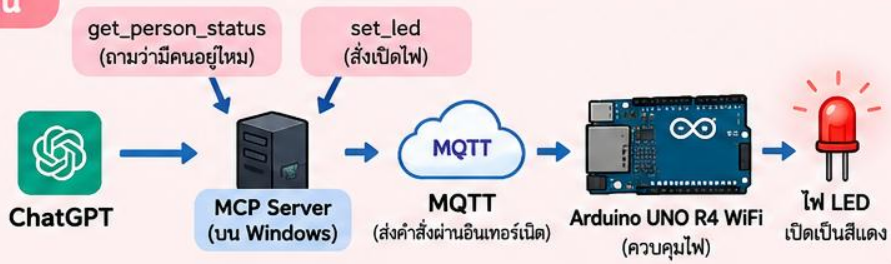
4 อุปกรณ์ทำงาน



เช่น สั่งผ่าน MQTT
ไปยัง Arduino ให้เปิดไฟสีแดง

ตัวอย่างในโปรเจกต์จริงของคุณ





ทำไมต้องใช้ MCP?

- สั่งงานได้ด้วยภาษาธรรมดา
  ไม่ต้องเขียนโค้ดเอง
- เชื่อมต่อกับอุปกรณ์และโปรแกรมได้หลายแบบ
- เป็นมาตรฐาน ทำให้โปรแกรมต่าง ๆ
  นำไปใช้ร่วมกันได้
- ปลอดภัย ควบคุมได้ว่า AI ทำอะไรได้บ้าง
- เหมาะสำหรับงานจริง เช่น บ้านอัจฉริยะ,
  โรงงาน, หุ่นยนต์, ระบบอัตโนมัติ

MCP (Model Context Protocol)

เชื่อม AI กับข้อมูลและเครื่องมือต่าง ๆ อย่างปลอดภัย

MCP เป็นมาตรฐานการสื่อสารระหว่าง AI Client และ MCP Server เพื่อให้ AI สามารถเข้าถึงเครื่องมือ (Tools) ข้อมูล (Resources) และแม่แบบคำสั่ง (Prompts) จากระบบภายนอกได้อย่างปลอดภัย

Tools (เครื่องมือสำหรับลงมือทำ)

- ใช้สั่งให้โปรแกรมทำงานหรือควบคุมอุปกรณ์
- รองรับการทำงานที่มีผลเปลี่ยนแปลง (Write/Action)
- AI เลือกเรียกใช้ตามความเหมาะสม

ตัวอย่าง

```
set_led(action: "on" | "off")  # เปิด/ปิด LED
get_person_status()            # ตรวจจับคน
publish_mqtt(topic, message)   # ส่งข้อความ MQTT
```

Resources (แหล่งข้อมูล)

- เปิดเผยข้อมูลให้ Client อ่าน
- เหมาะกับข้อมูลที่เปลี่ยนบ่อยหรือดึงมาอ่านอย่างเดียว
- ระบุด้วย URI (ใช้ scheme อะไรก็ได้)

ตัวอย่าง

```
iot://sensors/temperature     # ค่าอุณหภูมิ
iot://sensors/humidity        # ค่าความชื้น
iot://camera/person-status    # สถานะกล้อง
file://logs/latest            # ไฟล์ log ล่าสุด
```

Prompts (แม่แบบคำสั่งสำหรับ AI)

- เตรียมข้อความหรือโครงสร้าง Prompt
- ช่วยให้ผู้ใช้เริ่มงานได้ง่าย และได้ผลลัพธ์เป็นรูปแบบเดียวกัน
- Client ขอ Prompt จาก Server แล้วนำไปให้ AI ใช้

ตัวอย่าง

```
analyze_iot_status(status: str)
  # เนื้อหาเพื่อวิเคราะห์สถานะ IoT
summarize_log(log_text: str)
  # เนื้อหาสรุป log
create_report(data: str)
```

สถาปัตยกรรมและลำดับการทำงาน



วิธีการเรียกใช้จาก MCP Client (ตัวอย่าง Python)

เรียกใช้ Tool
สั่งให้ทำงาน

```python
result = await session.call_tool(
    "set_led",
    {"action": "on"}
)
print(result)
```

ใช้เมื่อต้องการสั่งให้เกิดการทำงาน เช่น เปิด/ปิด LED

อ่าน Resource
ขอข้อมูลจาก URI

```python
result = await session.read_resource(
    "iot://sensors/temperature"
)
for item in result.contents:
    print(item.text)
```

ใช้เมื่อต้องการอ่านข้อมูล เช่น ค่าอุณหภูมิ ค่าความชื้น

ขอ Prompt
รับแม่แบบคำสั่ง

```python
result = await session.get_prompt(
    "analyze_iot_status",
    {"status": "Temperature: 28.5 C"}
)
for msg in result.messages:
    print(msg.content)
```

ใช้เมื่อต้องการให้ Server สร้างข้อความสำหรับให้ AI ใช้ เช่น วิเคราะห์สถานะระบบ

Transport (วิธีการเชื่อมต่อ)

1. stdio

- ใช้การสื่อสารผ่าน Standard Input/Output
- เหมาะกับการรันแบบ Local process
- ใช้งานง่าย ไม่ต้องมี Web server

MCP Client (กระบวนการเดียวกัน) ↕ MCP Server (stdio)

2. Streamable HTTP

- ใช้ HTTP(S) สำหรับการสื่อสาร
- เหมาะกับการใช้งานผ่านเครือข่าย หรือให้บริการแบบ remote
- รองรับการสตรีมข้อมูล (streaming)

MCP Client (ผ่านอินเทอร์เน็ต/เครือข่าย) ↕ HTTPS (Streamable HTTP) ↕ MCP Server (บนเซิร์ฟเวอร์)

Authentication และ Authorization

Authentication (การยืนยันตัวตน)

- ตรวจสอบว่า Client เป็นใคร
- ตัวอย่างวิธีการ
  - API Key (ส่งใน HTTP Header)
  - OAuth 2.0 / OpenID Connect
  - Client Certificate (mTLS)

ตัวอย่าง HTTP Header (API Key)

```
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json
```

Authorization (การอนุญาต)

- ตรวจสอบว่า Client มีสิทธิ์ทำอะไรได้บ้าง
- กำหนดการเข้าถึงตามบทบาท หรือขอบเขตสิทธิ์ (Scope/Role)
- ตัวอย่าง
  - อนุญาตให้เรียกเฉพาะ Tool บางตัว
  - อนุญาตให้เข้าถึง Resource บาง URI
  - จำกัดสิทธิ์ตามผู้ใช้หรือแอปพลิเคชัน

ลำดับการทำงานของ MCP

- MCP Server แจ้ง AI ว่ามี Tool อะไรให้ใช้บ้าง
- เมื่อ AI Chatbot ได้รับภาษาธรรมชาติจาก User ก็จะเลือกว่าจะเรียกใช้ Tool อะไร โดยอาศัยคำอธิบายของ Tool ที่ MCP Server ส่งมาให้ตอนแรก เช่น

```python
@mcp.tool()
def set_led(action: str) -> str:
    """Turn the IoT LED on or off."""
```

MCP Server ประกาศข้อมูลให้ AI ทราบว่า

- ชื่อเครื่องมือคือ `set_led`
- ใช้สำหรับเปิดหรือปิด LED
- ต้องส่งพารามิเตอร์ชื่อ action
- ค่า action เป็นข้อความ เช่น on หรือ off

AI จึงรู้ว่าเครื่องมือนี้ทำอะไร และต้องส่งข้อมูลรูปแบบใด

ลำดับการทำงานของ MCP

- เมื่อ AI Chatbot ได้รับภาษาธรรมชาติจาก User จะทำการวิเคราะห์ความหมาย เช่น
  - ผู้ใช้ต้องการเปิดไฟ จึงควรจะเรียกเครื่องมือ set_led และกำหนด action = "on“

- ข้อสังเกต : การเลือก Tool จะเป็นการตีความของ LLM ไม่ใช่การจับคู่ประโยคแบบตายตัว ซึ่ง AI อาจจะเลือกผิดได้ ถ้าคำอธิบายไม่ชัดเจน หรือคำขอจาก user มีความกำกวม

- สรุป
  - MCP Server ประกาศเครื่องมือที่มี
  - AI เข้าใจคำสั่งของผู้ใช้
  - AI เลือก Tool
  - MCP Client ส่งคำขอไปยัง MCP Server
  - MCP Server เรียกใช้ฟังก์ชันเพื่อควบคุมอุปกรณ์
  - อุปกรณ์ทำงานจริง

ลำดับการทำงานของ MCP

ตัวอย่าง: สั่งเปิด LED ด้วยภาษาธรรมชาติ

1 MCP Server ประกาศ Tool ที่มี

ผมมี Tool ให้ใช้ ดังนี้
- set_led(action)
- get_person_status()
...



MCP Server

ตัวอย่างการประกาศ Tool ใน server.py

```
@mcp.tool()
def set_led(action: str):
 """Turn the LED on or off."""

@mcp.tool()
def get_person_status():
  """Get the person detection
  status."""
```

2 AI เข้าใจคำสั่งของผู้ใช้

ช่วยเปิดไฟ LED หน่อย



ผู้ใช้ (ภาษาธรรมชาติ)

ChatGPT (AI)

เข้าใจว่าผู้ใช้ต้องการเปิดไฟ LED

3 AI เลือก Tool ที่เหมาะสม

ควรเรียกใช้เครื่องมือ set_led และกำหนด action = "on"



Tool Call ที่ AI สร้าง

```
{
"tool1": "set_led",
 "arguments": {
   "action": "on"
}
```

4 MCP Client ส่งคำขอไปยัง MCP Server



MCP Client (ใน ChatGPT)

ส่งคำขอเรียกใช้ Tool

ส่งผลลัพธ์กลับมา

5 MCP Server เรียกใช้ฟังก์ชันเพื่อควบคุมอุปกรณ์



MCP Server (โปรแกรม Python)

ตัวอย่างโค้ดใน server.py

```
def set_led(action: str):
  if action == "on":
    # กรณี 1 : ควบคุม GPIO
    GPIO.output(LIGHT, True)

    # หรือ กรณี 2 : ส่งผ่าน MQTT
    mqtt_client.publish(
      "iot/led",
      "on"
    )
```

6 อุปกรณ์ทำงานจริง

กรณี 1: ควบคุม GPIO ที่ Raspberry Pi โดยตรง



Raspberry Pi เปิด LED ผ่าน GPIO

กรณี 2: ส่งคำสั่งผ่าน MQTT ไปยัง Arduino



Arduino รับข้อความ MQTT แล้วเปิด LED



การทดลอง

Natural Language AI Control

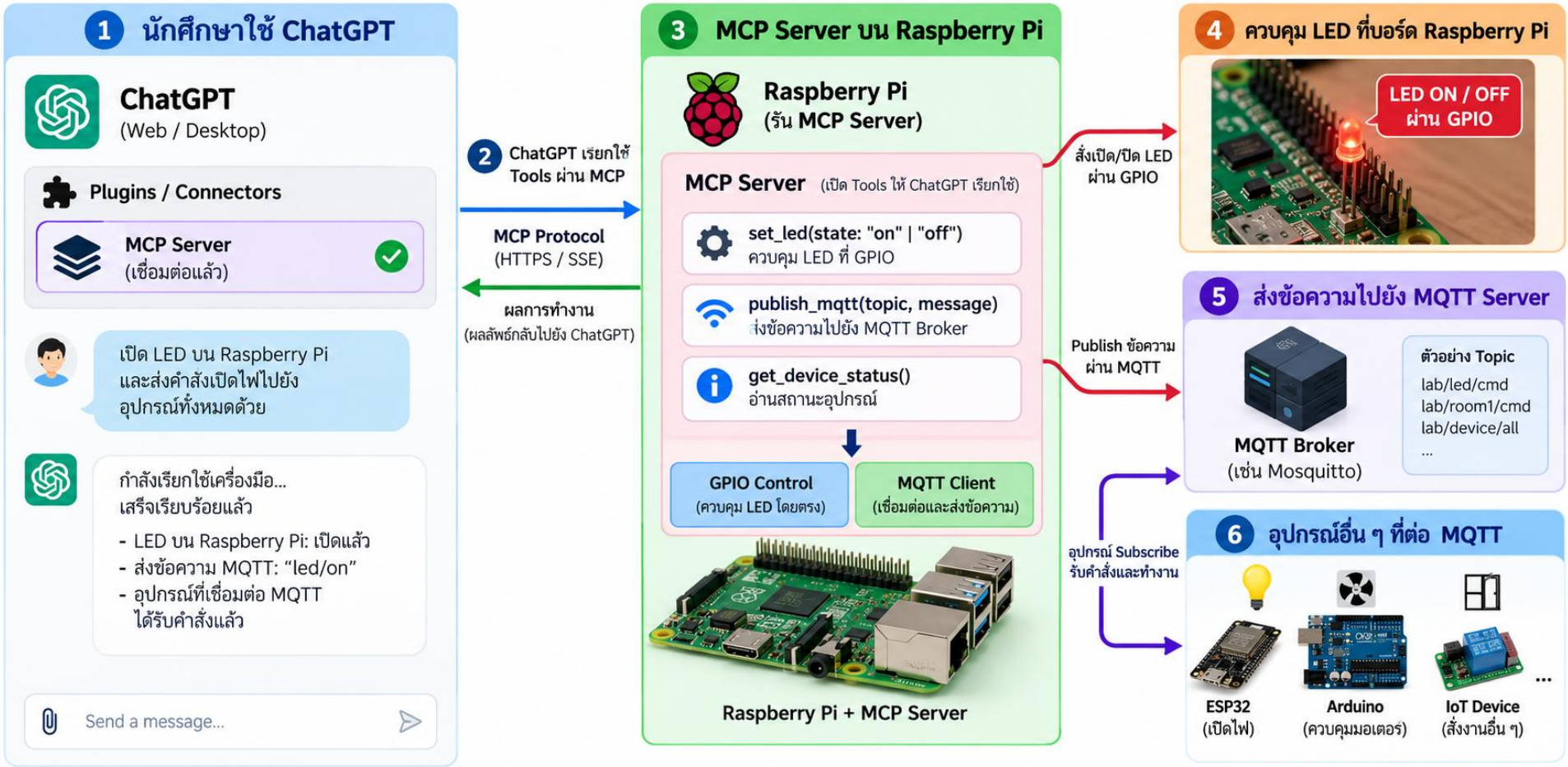


ทดลองใช้งาน AI & IoT โดยใช้ MCP Server

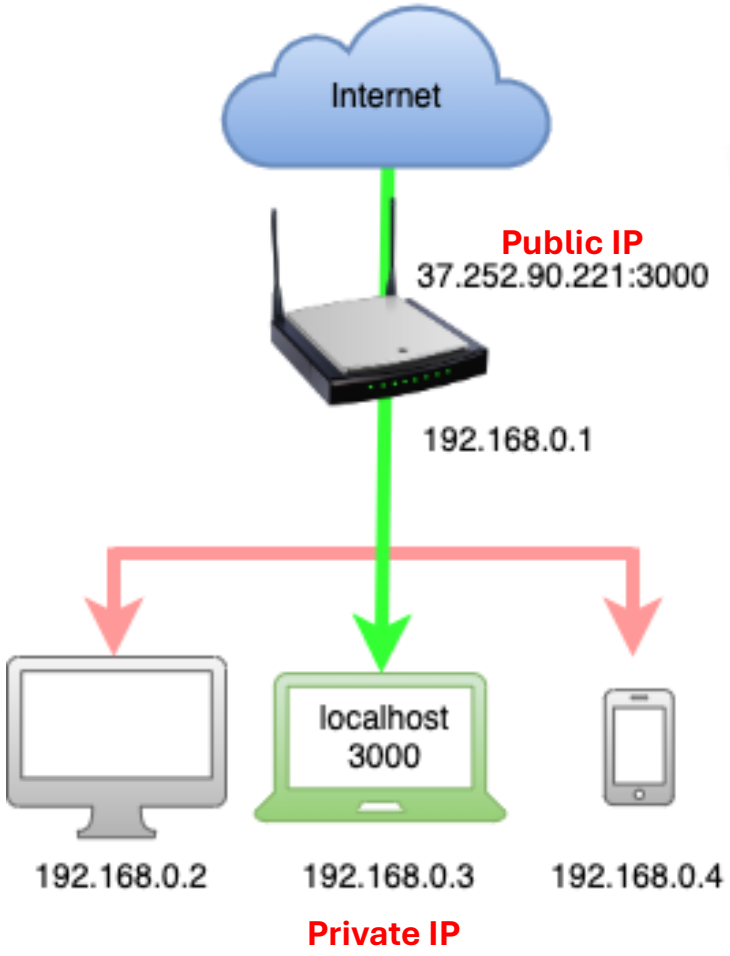
ขั้นตอน

- ติดตั้ง MCP Server บน Raspberry Pi
- สร้าง Tunnel เพื่อให้ AI สามารถเข้าถึงได้ผ่าน Internet
- เขียน/แก้ไข Code python เพื่อสร้าง Tool สำหรับให้ AI เข้ามาเรียกใช้งาน
- Run code ที่เขียนเสร็จแล้ว
- ต่อ LED บน Raspberry Pi หรือ Subscribe MQTT Topic เพื่อรอรับค่า
- เชื่อม Generative AI Chatbots เข้ากับ MCP Server ที่ตั้งไว้
  - ChatGPT Go ขึ้นไป
  - ai.kmitl.ac.th
- ทดลองพิมพ์คำสั่งบน AI Chat Input (Prompt Box) เช่น เปิดไฟหน่อย
- สังเกตผลลัพธ์ที่ได้ เช่น LED ติด/ดับ หรือ ได้รับค่าจาก MQTT Broker

ภาพรวมการทดลอง



Tunneling Technologies





การ Access localhost จาก Internet

- Port Forwarding at Router
- Port Forwarding Service
  - Ngrok (https://ngrok.com/)
  - Localhost.run (https://localhost.run/)
  - Tunnl.gg (https://tunnl.gg/)

https://tunnl.gg/

ข้อดี
ใช้งานง่าย ไม่ต้องสมัครสมาชิก
ข้อเสีย
URL จะเปลี่ยนทุกครั้งที่รัน (เว้นแต่ใช้แผนแบบจ่ายเงิน)

- คือ บริการออนไลน์ ที่ทำให้เปิดพอร์ตจากเครื่อง local ออกอินเทอร์เน็ต ได้ ผ่าน SSH tunneling
- ทำให้ server ที่รันอยู่ในเครื่อง (เช่น web app บน localhost:8000) เข้าถึงได้จากภายนอก โดยไม่ต้องตั้งค่า router หรือ firewall
- **วิธีทำงาน**
  - เปิด Terminal ขึ้นมาใหม่ แล้วรันคำสั่งใน terminal ดังนี้
    - `ssh-keygen -t ed25519`
    - `ssh -t -R 80:127.0.0.1:8000 stable@proxy.tunnl.gg`

    คำอธิบาย
    - `-R` = reverse tunnel
    - `80:127.0.0.1:8000` หมายถึง ให้ forward พอร์ต 80 ของเซิร์ฟเวอร์ tunnl.gg มาที่ 127.0.0.1:8000 บนเครื่องของเรา
    - tunnl.gg จะสร้าง Public URL ให้ทุกคนสามารถเข้าถึงได้
    - เช่น : ruby-mountain-41a2c079.tunnl.gg
    - ใครก็ตามที่เปิด URL นี้ จะถูกส่ง request มายังเครื่องของเรา ผ่าน SSH tunnel

ขั้นตอนการทดลอง 1. ติดตั้ง Library MCP

```
$ sudo apt update
$ sudo apt install python3-pip
```

สร้างโฟลเดอร์โปรเจกต์

```
$ mkdir -p ~/iot-mcp
$ cd ~/iot-mcp
```

สร้าง virtual environment:

```
$ sudo apt install python3-full python3-venv -y
$  python3 -m venv ~/iot-mcp
$  source ~/iot-mcp/bin/activate
```

ติดตั้ง Library

```
(iot-mcp) $ pip3 install "mcp[cli]" paho-mqtt
(iot-mcp) $ pip3 install RPi.GPIO
```

ตัวอย่าง ผลการ run คำสั่ง

```
pi@panwit:~ $ sudo apt install python3-pip
python3-pip is already the newest version (25.1.1+dfsg-1+rpt1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 129
pi@panwit:~ $ mkdir -p ~/iot-mcp
pi@panwit:~ $ ls
coap-env  coap_server.py  iot-mcp  led.py
pi@panwit:~ $ cd ~/iot-mcp
pi@panwit:~/iot-mcp $ sudo apt install python3-full python3-venv -y
python3-full is already the newest version (3.13.5-1).
python3-venv is already the newest version (3.13.5-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 129
pi@panwit:~/iot-mcp $ python3 -m venv ~/iot-mcp
pi@panwit:~/iot-mcp $ source ~/iot-mcp/bin/activate
(iot-mcp) pi@panwit:~/iot-mcp $ pip3 install "mcp[cli]" paho-mqtt
```

ขั้นตอนการทดลอง 2. ติดตั้ง tunnel และ แก้ code

ติดตั้ง tunnel (เนื่องจาก Raspi ได้ Private IP จึงต้องประกาศ Public URL)

เปิด Terminal ขึ้นมาใหม่ แล้วรันคำสั่งใน terminal ดังนี้

```
$ ssh-keygen -t ed25519
$ ssh -t -R 80:127.0.0.1:8000 stable@proxy.tunnl.gg
```

ตัวอย่าง การ run https://tunnl.gg/

```
pi@panwit:~ $ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/pi/.ssh/id_ed25519):
Enter passphrase for "/home/pi/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/pi/.ssh/id_ed25519
Your public key has been saved in /home/pi/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:uajp6Vtiy2sPwmztGxhmkQutDBy7zm4U8bj30/aBcIE pi@panwit
The key's randomart image is:
+--[ED25519 256]--+
| .               |
|.oo.   .         |
|ooB  E .         |
|o=.+    ..       |
|.oB  . .S        |
|oB =  o...       |
|.oB B +...       |
|.o =.% o  .      |
|.. +&=+ ..       |
+----[SHA256]-----+
pi@panwit:~ $
```

ตัวอย่าง การ run https://tunnl.gg/

```
pi@panwit:~ $ ssh -t -R 80:127.0.0.1:8000 stable@proxy.tunnl.gg
The authenticity of host 'proxy.tunnl.gg (46.62.160.120)' can't be established.
ED25519 key fingerprint is SHA256:5uQ6VobmUpY4wyMmm5jnAwFtt3aX06WYDDM4ER+2WUw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'proxy.tunnl.gg' (ED25519) to the list of known hosts
.

  • Tunnel is live on tunnl.gg

  URL      https://gentle-wave-62fcc6af.tunnl.gg
           stays the same for your SSH key
  Expires  in 24h, or after 2h without traffic
           Pro: your own name and no time limits, from $5/mo: https://tunnl.gg/p
ricing
```

ขั้นตอนการทดลอง 2. ติดตั้ง tunnel และ แก้ code

แก้ไข Code และ Run Code

นำค่า tunnel.gg ที่ได้มาใส่ใน code

```
(iot-mcp) $ nano server.py
(iot-mcp) $ python3 server.py
```

ตัวอย่าง ผลการทดสอบเข้า raspi จาก Internet

- เปิด Web Browser แล้วเรียก URL ตามที่ได้จาก tunnl.gg
- พิมพ์ URL ที่ได้ตามด้วย /mcp
- จะขึ้นหน้าจอ เตือน ขึ้นมาก่อน ก็ให้ Continues จะแสดงดังรูป



ตัวอย่าง code MCP Server ส่วนสร้าง tool

```python
@mcp.tool()
def set_led(action: str) -> str:
    """Turn the IoT LED on or off.
    action must be exactly 'on' or 'off'.
    """
    action = action.strip().lower()
    if action not in ("on", "off"):
        return "Error: action must be on or off"
    # Publish only an allowed command
    info = mqtt_client.publish(COMMAND_TOPIC,action.upper(),
        qos=1
    )
    info.wait_for_publish(timeout=5)
    if info.rc != mqtt.MQTT_ERR_SUCCESS:
        return "MQTT publish failed"
    return f"Sent LED command: {action.upper()}"
```

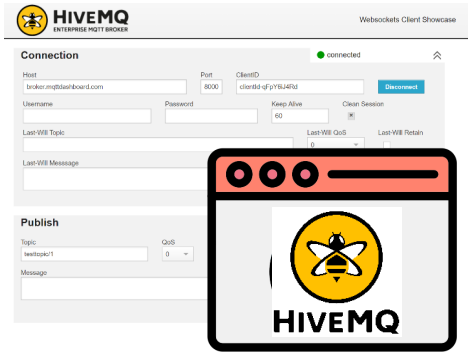
ชื่อ Tool คือ set_led
ใช้สำหรับเปิดหรือปิด LED
ต้องส่งพารามิเตอร์ชื่อ action
ค่า action เป็นข้อความ เช่น on หรือ off

ภาพรวมขั้นตอนการทดลอง. MCP with Raspberry Pi

- นำมือถือของ นศ เปิด Hot Spot ตั้งค่า SSID, Password
- ติดตั้ง Library , สร้าง Tunnel
- แก้ค่า Client ID, Device ID , **tunnel.gg** ในโปรแกรม ให้ถูกต้อง
- ทำการ Run Code ที่แก้ไขแล้ว
- ต่อ LED เข้าที่ GPIO 14 , Subscribe Topic ตาม Device ID
- เชื่อม AI Chatbot เข้ากับ MCP Server
- พิมพ์สั่งทาง Chat Input
- สังเกตผลการทดลอง









MCP Server

Code ตัวอย่าง server.py

แก้ Code ในส่วนต่อไปนี้

server.py

แก้ Device ID

แก้ Client ID

แก้ tunnel.gg

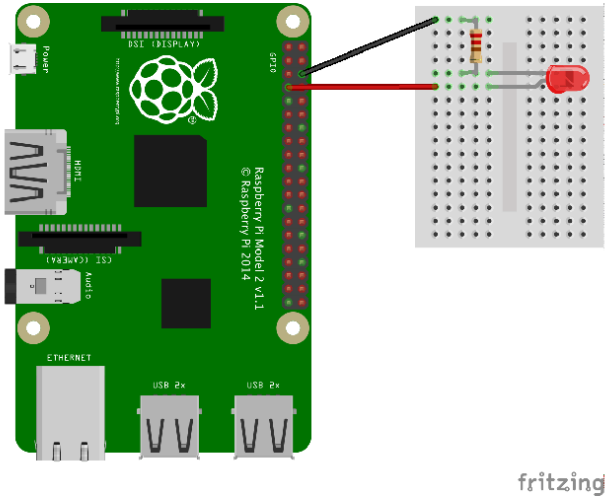
```
# MQTT connection
mqtt_client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="raspi-iot-mcp-panwit"
)
```

```
DEVICE_ID = "personlight-demo"
COMMAND_TOPIC = f"{DEVICE_ID}/command"
STATUS_TOPIC = f"{DEVICE_ID}/person"
```

```
if __name__ == "__main__":
    # MCP over Streamable HTTP
    security = TransportSecuritySettings(
        allowed_hosts=[
            "gentle-wave-62fcc6af.tunnl.gg",
            "gentle-wave-62fcc6af.tunnl.gg:*",
            "localhost:8000",
            "127.0.0.1:8000",
            "0.0.0.0:8000",
        ],
        allowed_origins=[
            "https://gentle-wave-62fcc6af.tunnl.gg/",
    ],
    )
```

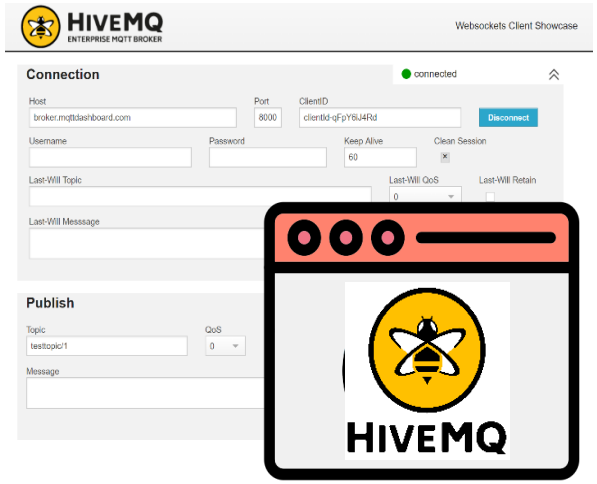
ขั้นตอนการทดลอง 3. ต่อ LED , Subscribe Topic

- ต่อ LED เข้ากับขา GPIO4

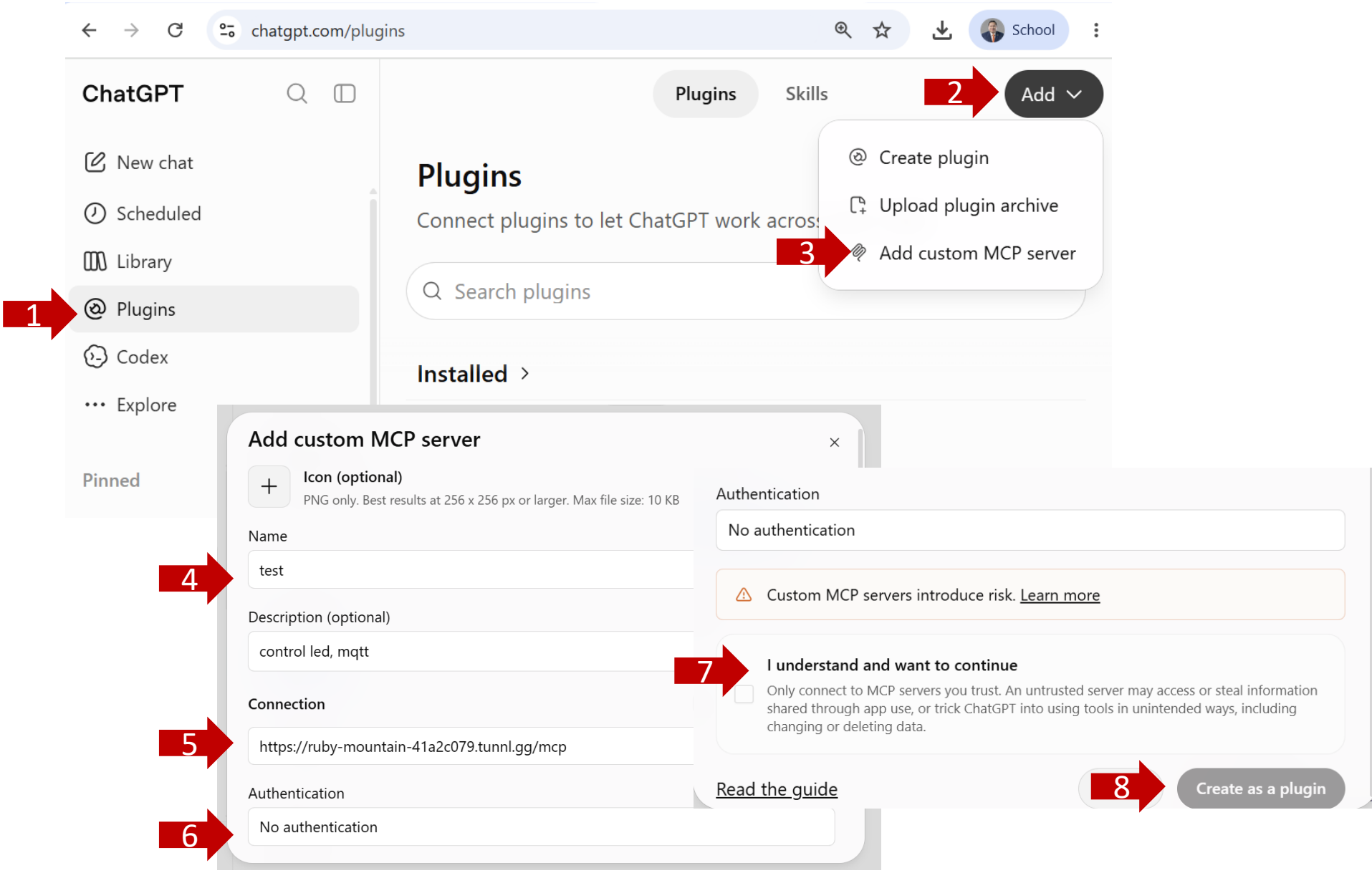


- ใช้ Web Browser เชื่อมต่อกับ MQTT Broker `mqtt-dashboard.com`

  และ Subscribe ที่ Topic `{DEVICE_ID}/command`



ขั้นตอนการทดลอง 4. เชื่อม AI Chatbot กับ MCP Server



ทดสอบ prompt ไปว่า @test1 มี tool ไรบ้าง

Plugin `@test1` มีทั้งหมด **3 Tools** สำหรับเชื่อมต่อ AI กับระบบ IoT ได้แก่

Tool	หน้าที่
`mcp__test1__set_led`	สั่งเปิด/ปิด LED ของอุปกรณ์ IoT
`mcp__test1__set_led_raspi`	สั่งเปิด/ปิด LED ที่ Raspberry Pi
`mcp__test1__get_person_status`	อ่านสถานะล่าสุดจากระบบตรวจจับบุคคลด้วยกล้อง

วิธีเรียกใช้งาน

1. เปิด LED ของ IoT

JSON

```
{"action": "on"}
```

ตัวอย่างผลลัพธ์ที่ได้

จาร์วิส ปิดไฟละกัน

Worked for 4s

รับทราบครับ! จาร์วิสสั่งปิดไฟให้แล้วครับ

ปิดไฟแล้วครับ!

ส่งคำสั่งผ่าน MCP สำเร็จ

Tool: `mcp__test__set_led`

Action: `off`

ผลลัพธ์จริง: `Sent LED command: OFF`

ไฟดับหรือยังครับ? ผมส่งคำสั่งแล้ว แต่ยังไม่มีข้อมูลยืนยันจากเซนเซอร์ที่อุปกรณ์จริงครับ

ตัวอย่างผลลัพธ์ที่ได้

จาร์วิส เปิดไฟให้หน่อย

Worked for 4s

รับทราบครับ! ⚡ จาร์วิสสั่งเปิดไฟให้แล้วครับ

เปิดไฟแล้วครับ!

ส่งคำสั่งผ่าน MCP สำเร็จ

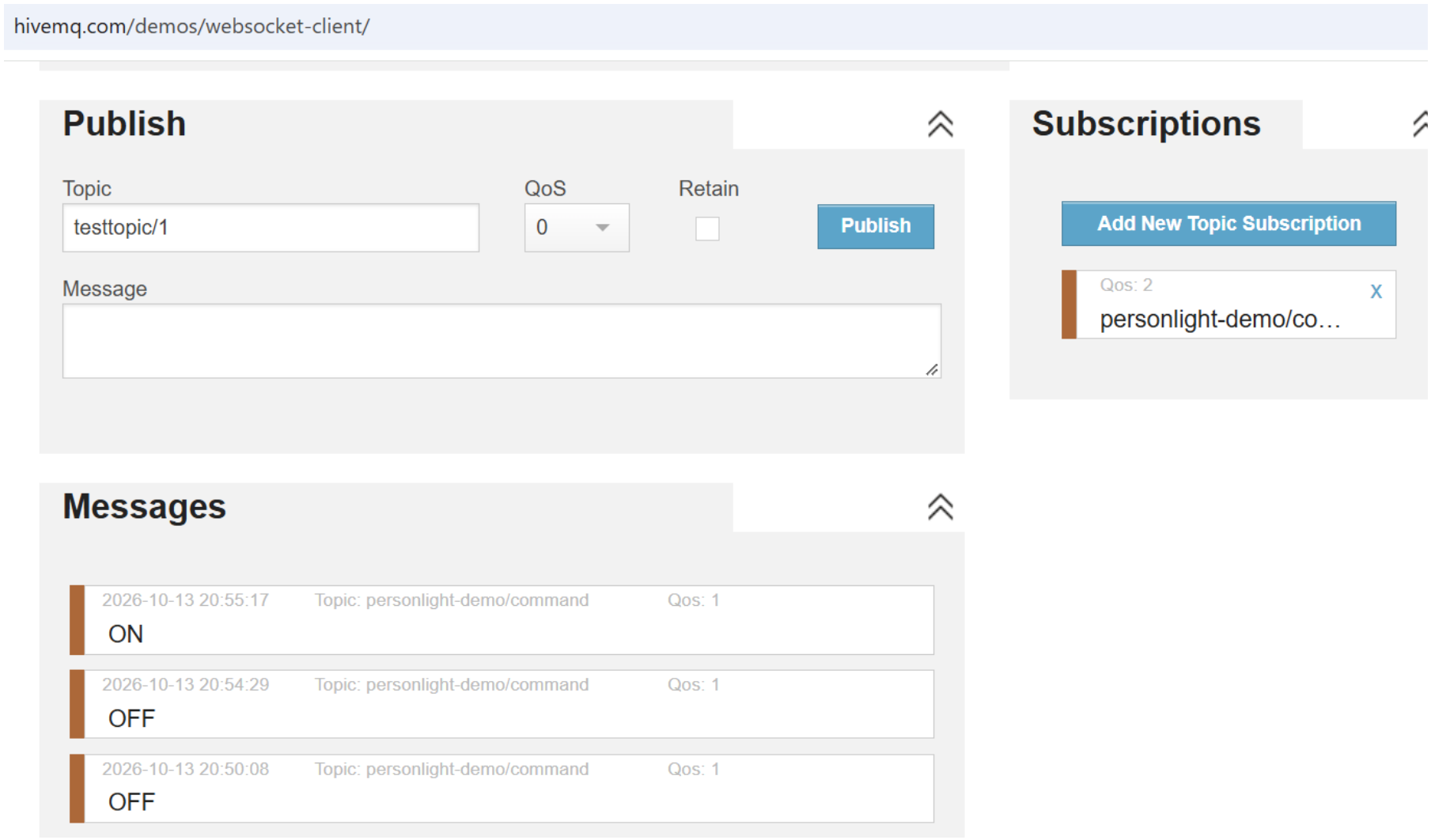
Tool: `mcp__test__set_led`

Action: `on`

ผลลัพธ์จริง: `Sent LED command: ON`

รับคำสั่งครับ! 😄 แต่ต้องตรวจสอบที่อุปกรณ์จริงอีกครั้งเพื่อยืนยันว่า LED ติดแล้วครับ

ตัวอย่างผลลัพธ์ที่ได้



ตัวอย่างผลลัพธ์ที่ได้

เปิดไฟที่ raspi อีกที

Worked for 4s

เปิดไฟ LED ที่ Raspberry Pi สำเร็จแล้วครับ! ✅

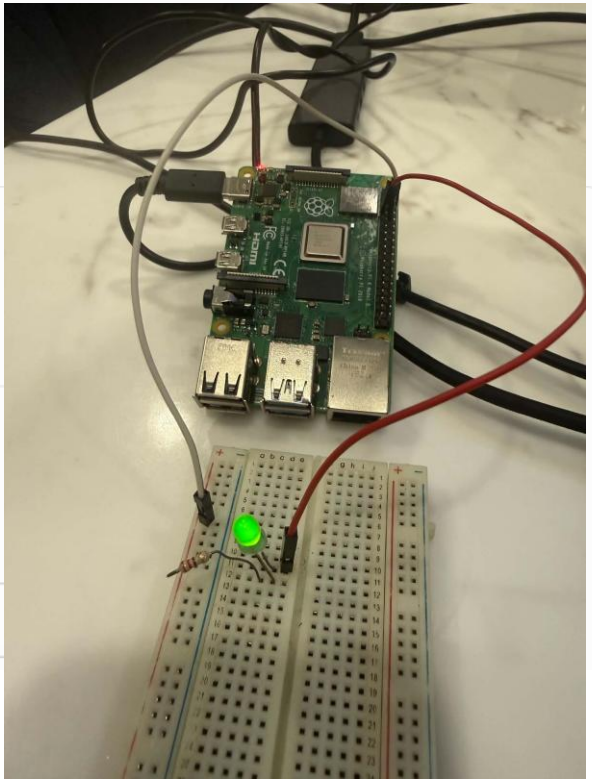
Raspberry Pi LED

ON — เปิดไฟแล้ว

ผลตอบกลับจาก MCP Tool `mcp__test1__set_led_raspi`

`LED is ON`

ปิดไฟ



ตัวอย่างผลลัพธ์ที่ได้

งั้นลองปิด

Worked for 4s

ปิดไฟ LED ที่ Raspberry Pi สำเร็จแล้วครับ! ✅

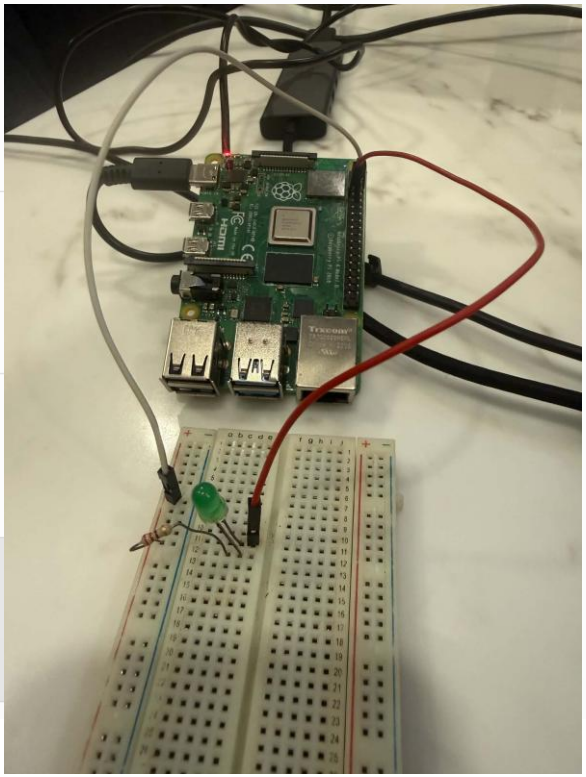
Raspberry Pi LED

OFF — ปิดไฟแล้ว

ผลตอบกลับจริงจาก MCP Tool `mcp__test1__set_led_raspi`:

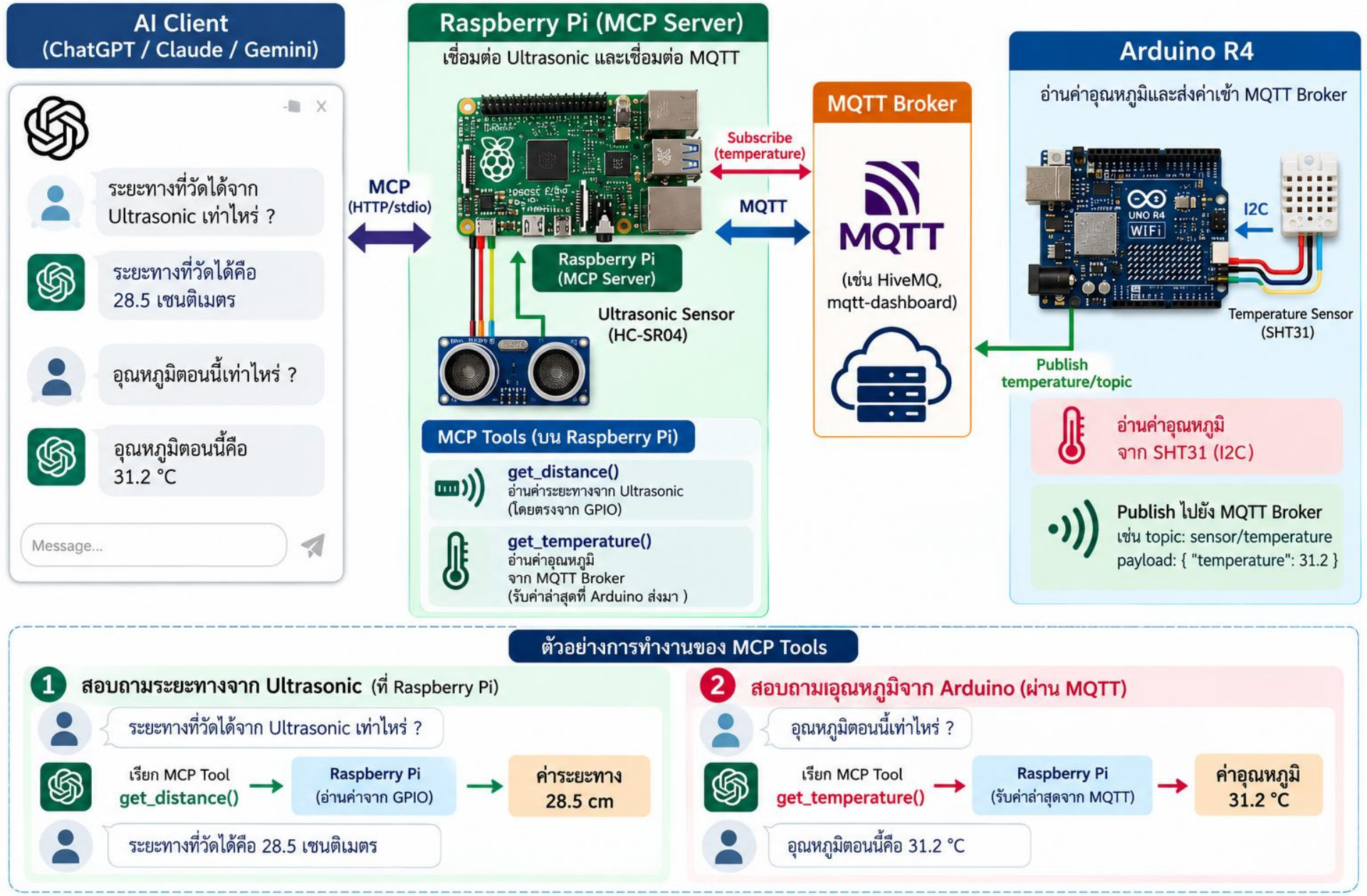
Plain text

```
LED is OFF
```



แบบฝึกหัด

- ต่อ Ultrasonic เข้ากับ Board Raspberry Pi
- ต่อ Sensor อุณหภูมิ เข้ากับ Arduino R4 และต่อเข้า MQTT Broker
- ให้สร้าง tool ใน MCP Server เพื่อรองรับ AI Model ในการ
  - สอบถามค่า ระยะทาง ที่วัดได้จาก Ultrasonic
  - สอบถามค่า อุณหภูมิ ที่ได้จากการ Subscribe ค่า อุณหภูมิ จาก Arduino


AI Client
(ChatGPT / Claude / Gemini)
ระยะทางที่วัดได้จาก Ultrasonic เท่าไหร่ ?
ระยะทางที่วัดได้คือ 28.5 เซนติเมตร
อุณหภูมิตอนนี้เท่าไหร่ ?
อุณหภูมิตอนนี้คือ 31.2 °C
Message...
MCP
(HTTP/stdio)
Raspberry Pi (MCP Server)
เชื่อมต่อ Ultrasonic และเชื่อมต่อ MQTT
Raspberry Pi
(MCP Server)
Ultrasonic Sensor
(HC-SR04)
Subscribe
(temperature)
MQTT
MQTT Broker
MQTT
(เช่น HiveMQ,
mqtt-dashboard)
Publish
temperature/topic
Arduino R4
อ่านค่าอุณหภูมิและส่งค่าเข้า MQTT Broker
UNO R4
WiFi
I2C
Temperature Sensor
(SHT31)
อ่านค่าอุณหภูมิ
จาก SHT31 (I2C)
Publish ไปยัง MQTT Broker
เช่น topic: sensor/temperature
payload: { "temperature": 31.2 }
MCP Tools (บน Raspberry Pi)
get_distance()
อ่านค่าระยะทางจาก Ultrasonic
(โดยตรงจาก GPIO)
get_temperature()
อ่านค่าอุณหภูมิ
จาก MQTT Broker
(รับค่าล่าสุดที่ Arduino ส่งมา)
ตัวอย่างการทำงานของ MCP Tools
1 สอบถามระยะทางจาก Ultrasonic (ที่ Raspberry Pi)
ระยะทางที่วัดได้จาก Ultrasonic เท่าไหร่ ?
เรียก MCP Tool
get_distance()
Raspberry Pi
(อ่านค่าจาก GPIO)
ค่าระยะทาง
28.5 cm
ระยะทางที่วัดได้คือ 28.5 เซนติเมตร
2 สอบถามเอุณหภูมิจาก Arduino (ผ่าน MQTT)
อุณหภูมิตอนนี้เท่าไหร่ ?
เรียก MCP Tool
get_temperature()
Raspberry Pi
(รับค่าล่าสุดจาก MQTT)
ค่าอุณหภูมิ
31.2 °C
อุณหภูมิตอนนี้คือ 31.2 °C


END